

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP LỚN
MÔN HỌC MÁY

Dự đoán mức thu nhập

Giáo viên hướng dẫn: Trường Vĩnh Lân

Lớp: L01

Nhóm: 10

Sinh viên thực hiện

Họ và tên	MSSV	Ghi chú
Trần Gia Hân	2052464	Nhóm trưởng <i>han.tran2202@hcmut.edu.vn</i>
Lê Minh Mẫn	2312040	<i>man.le9905@hcmut.edu.vn</i>
Nguyễn Thành Trình	2313640	<i>trinh.nguyen020705@hcmut.edu.vn</i>
Ngô Quang Tân	2313052	<i>tan.ngo196@hcmut.edu.vn</i>
Lương Mạnh Tiến	2213459	

TP. HỒ CHÍ MINH, 5/2026

Mục lục

Phân công công việc	iii
1 Giới thiệu đề tài	1
1.1 Giới thiệu bài toán dự đoán mức thu nhập	1
1.2 Mục tiêu của bài tập lớn	2
1.2.1 Xây dựng một pipeline học máy hoàn chỉnh	2
1.2.2 So sánh nhiều mô hình trên cùng một nền dữ liệu	2
1.2.3 Phân tích tác động của tiền xử lý và biểu diễn dữ liệu	2
1.2.4 Đánh giá tính phù hợp của học sâu trên dữ liệu bảng	3
1.2.5 Tăng khả năng diễn giải và tái sử dụng kết quả	3
2 Dữ liệu và phân tích khám phá dữ liệu	4
2.1 Tổng quan bộ dữ liệu	4
2.2 Cấu trúc dữ liệu và các nhóm thuộc tính	4
2.3 Phân tích khám phá dữ liệu	5
2.3.1 Phân bố của biến mục tiêu	5
2.3.2 Khảo sát giá trị thiếu	5
2.3.3 Phân tích các biến số	6
2.3.4 Phân tích các biến phân loại	6
2.3.5 Phát hiện đặc trưng dư thừa	6
2.4 Nhận xét rút ra từ giai đoạn EDA	6
3 Tiền xử lý dữ liệu	8
3.1 Mục tiêu và nguyên tắc thiết kế pipeline	8
3.2 Làm sạch dữ liệu ban đầu	8
3.2.1 Chuẩn hóa biểu diễn giá trị thiếu	8
3.2.2 Loại bỏ đặc trưng dư thừa	9
3.2.3 Tách đặc trưng và biến mục tiêu	9
3.3 Phân nhóm thuộc tính và chia dữ liệu	9
3.3.1 Xác định biến số và biến phân loại	9
3.3.2 Chia train/test có kiểm soát	9
3.4 Xây dựng các cấu hình preprocessing cho nhánh học máy truyền thống	10
3.4.1 Thiết kế bằng <i>Pipeline</i> và <i>ColumnTransformer</i>	10
3.4.2 Các cấu hình	10
3.4.3 So sánh số chiều đầu ra	11
3.5 Đánh giá và lựa chọn cấu hình tốt nhất	11
3.5.1 Chiến lược đánh giá	11
3.5.2 Kết quả so sánh các cấu hình	11

3.5.3	Lựa chọn đầu ra chuẩn cho các chương tiếp theo	12
3.6	Kết luận chương	12
4	Pipeline học máy truyền thống	13
4.1	Trích xuất đặc trưng	13
4.2	Huấn luyện mô hình baseline và đánh giá sơ bộ	14
4.2.1	Thiết lập huấn luyện Baseline	14
4.2.2	Kết quả thực nghiệm	14
4.2.3	Phân tích và đánh giá sơ bộ	14
4.2.4	Phân tích chi tiết mô hình Baseline tốt nhất	15
4.3	Tinh chỉnh siêu tham số	16
4.3.1	Chiến lược tối ưu hóa	16
4.3.2	Kết quả tinh chỉnh chi tiết	17
4.3.3	Đánh giá qua trực quan hóa	17
4.4	So sánh và lựa chọn mô hình cuối cùng	19
4.5	Phân tích mức độ quan trọng của đặc trưng	20
5	Pipeline học sâu	22
5.1	Giới thiệu tổng quan	22
5.1.1	Định hướng thiết kế	22
5.1.2	Kết quả kỳ vọng	22
5.2	Tiền xử lý dữ liệu học sâu	23
5.2.1	Làm sạch dữ liệu và chuẩn hóa đầu vào	23
5.2.2	Chia tập dữ liệu	24
5.2.3	Tiền xử lý cho MLP	24
5.2.4	Xử lý mất cân bằng lớp	25
5.2.5	Tiền xử lý riêng cho TabNet	25
5.3	Mạng nơ-ron học đặc trưng	26
5.3.1	MLP – Multi-Layer Perceptron	26
5.3.2	TabNet – kiến trúc chuyên biệt cho dữ liệu bảng	26
5.4	Huấn luyện đầu-cuối	26
5.4.1	Thiết lập thí nghiệm	26
5.4.2	Huấn luyện MLP cơ bản	27
5.4.3	Tối ưu MLP bằng Optuna	28
5.4.4	Huấn luyện TabNet và pretraining	29
5.5	Đánh giá	30
5.5.1	Phương pháp đánh giá	30
5.5.2	Kết quả thực nghiệm	31
5.5.3	Triển khai mô hình	32
6	So sánh kết quả và phân tích	35
6.1	So sánh giữa pipeline truyền thống và pipeline học sâu	35
6.2	Ưu và nhược điểm của từng hướng tiếp cận	37
6.3	Kết luận chung	37

Phân công công việc

Họ và tên	MSSV	Nội dung phụ trách	Tỷ lệ đóng góp
Trần Gia Hân	2052464	Pipeline học máy truyền thống và viết báo cáo.	100%
Lê Minh Mẫn	2312040	Pipeline học sâu và viết báo cáo.	100%
Nguyễn Thành Trình	2313640	Pipeline học sâu và viết báo cáo.	100%
Ngô Quang Tân	2313052	EDA và viết báo cáo.	100%
Lương Mạnh Tiến	2213459	Tiền xử lý dữ liệu và viết báo cáo.	100%

Chương 1

Giới thiệu đề tài

1.1 Giới thiệu bài toán dự đoán mức thu nhập

Trong bối cảnh dữ liệu ngày càng được sử dụng rộng rãi để hỗ trợ ra quyết định, bài toán dự đoán mức thu nhập cá nhân là một ví dụ tiêu biểu cho hướng ứng dụng của học máy vào các vấn đề kinh tế – xã hội. Từ các thông tin nhân khẩu học và nghề nghiệp như độ tuổi, trình độ học vấn, tình trạng hôn nhân, nghề nghiệp, số giờ làm việc hay quốc gia xuất thân, mô hình có thể ước lượng khả năng một cá nhân thuộc nhóm thu nhập thấp hơn hoặc bằng 50K hay lớn hơn 50K mỗi năm. Đây là một bài toán phân loại nhị phân có ý nghĩa thực tiễn cao vì nó gắn với các vấn đề về lao động, phúc lợi xã hội, giáo dục và hoạch định chính sách.

Nhóm lựa chọn đề tài này vì đây là một bài toán vừa có tính ứng dụng, vừa có giá trị học thuật trong phạm vi môn học máy. Bộ dữ liệu đi kèm chứa đồng thời đặc trưng số và đặc trưng phân loại, có tồn tại giá trị thiếu, có hiện tượng mất cân bằng lớp và có cả những cột mang thông tin dư thừa. Vì vậy, đề tài không chỉ dừng ở việc huấn luyện một mô hình dự đoán, mà còn đòi hỏi nhóm phải giải quyết đầy đủ các bước của một pipeline học máy hoàn chỉnh: khảo sát dữ liệu, tiền xử lý, trích xuất đặc trưng, lựa chọn mô hình, tối ưu siêu tham số, đánh giá và so sánh các phương pháp.

Từ góc độ phương pháp, bài toán dự đoán thu nhập là một trường hợp rất phù hợp để so sánh giữa hai hướng tiếp cận lớn. Một mặt, các mô hình học máy truyền thống như Logistic Regression, Linear SVM, Random Forest hay Gaussian Naive Bayes thường hoạt động rất tốt trên dữ liệu bảng khi được tiền xử lý hợp lý. Mặt khác, các mô hình học sâu như MLP hay TabNet lại mở ra khả năng học các quan hệ phi tuyến phức tạp hơn giữa các thuộc tính đầu vào. Chính vì vậy, đề tài này cho phép nhóm không chỉ xây dựng một mô hình dự đoán, mà còn đánh giá sâu hơn câu hỏi: với dữ liệu bảng dạng điều tra dân số, phương pháp nào thật sự phù hợp hơn về cả hiệu năng lẫn chi phí triển khai.

Trong quá trình thực hiện, nhóm tiếp cận bài toán theo hướng thực nghiệm. Thay vì giả định ngay từ đầu rằng mô hình phức tạp sẽ cho kết quả tốt hơn, nhóm xây dựng từng pipeline, đối chiếu kết quả giữa các mô hình và phân tích vì sao một mô hình lại hoạt động hiệu quả hoặc kém hiệu quả hơn mô hình khác. Nhờ đó, báo cáo không chỉ trả lời “mô hình nào tốt nhất”, mà còn giải thích “vì sao mô hình đó phù hợp với bài toán”.

Một khía cạnh quan trọng khác của đề tài là khả năng diễn giải. Với bài toán thu nhập, kết quả dự đoán chỉ có ý nghĩa khi nhóm hiểu được những đặc trưng nào ảnh hưởng mạnh đến mô hình. Do đó, ngoài việc theo đuổi chỉ số đánh giá cao, nhóm còn quan tâm đến khả năng phân tích đặc trưng, so sánh giữa các cách biểu diễn dữ liệu và kiểm tra xem các kết luận của mô hình có phù hợp với trực giác từ dữ liệu hay không.

Tóm lại, đề tài dự đoán mức thu nhập là một bài toán tiêu biểu cho học máy trên dữ liệu

bảng. Đây là môi trường phù hợp để nhóm triển khai trọn vẹn một quy trình phân tích dữ liệu và huấn luyện mô hình, đồng thời tạo ra cơ sở rõ ràng để so sánh giữa pipeline học máy truyền thống và pipeline học sâu.

1.2 Mục tiêu của bài tập lớn

Bài tập lớn này được thực hiện với mục tiêu tổng quát là xây dựng và đánh giá một hệ thống dự đoán mức thu nhập cá nhân dựa trên bộ dữ liệu *Adult Census Income*, đồng thời so sánh hai hướng tiếp cận là học máy truyền thống và học sâu trên cùng một bài toán. Để đạt được mục tiêu tổng quát đó, nhóm triển khai đề tài theo các mục tiêu cụ thể sau.

1.2.1 Xây dựng một pipeline học máy hoàn chỉnh

Mục tiêu đầu tiên của nhóm là không chỉ huấn luyện một mô hình riêng lẻ, mà xây dựng một pipeline đầy đủ từ dữ liệu thô đến kết quả cuối cùng. Điều này bao gồm các bước: khám phá dữ liệu, phát hiện đặc trưng dư thừa, xử lý giá trị thiếu, mã hóa biến phân loại, chuẩn hóa biến số, chia tập dữ liệu, trích xuất đặc trưng, huấn luyện mô hình, tuning và đánh giá kết quả.

Nhóm đặt mục tiêu này vì trong các bài toán thực tế, chất lượng mô hình không chỉ phụ thuộc vào thuật toán mà còn phụ thuộc rất lớn vào chất lượng của quy trình xử lý dữ liệu trước đó. Một pipeline được tổ chức tốt cũng giúp việc tái lập thí nghiệm và kiểm chứng kết quả trở nên rõ ràng hơn.

1.2.2 So sánh nhiều mô hình trên cùng một nền dữ liệu

Mục tiêu thứ hai là so sánh hiệu năng của nhiều mô hình khác nhau trên cùng bộ dữ liệu và cùng chuẩn đánh giá. Ở nhánh học máy truyền thống, nhóm triển khai Logistic Regression, Linear SVM, Random Forest và Gaussian Naive Bayes. Ở nhánh học sâu, nhóm triển khai MLP và TabNet, đồng thời kết hợp thêm bước tối ưu siêu tham số bằng Optuna.

Thông qua việc so sánh này, nhóm muốn trả lời câu hỏi cốt lõi: mô hình nào phù hợp nhất với bài toán dự đoán mức thu nhập trên dữ liệu bảng? Việc đánh giá này không chỉ dựa trên một chỉ số đơn lẻ, mà dựa trên nhiều tiêu chí như Accuracy, Precision, Recall, F1-score và ROC-AUC để đảm bảo kết luận có tính toàn diện.

1.2.3 Phân tích tác động của tiền xử lý và biểu diễn dữ liệu

Một mục tiêu quan trọng khác là đánh giá vai trò của tiền xử lý dữ liệu đối với kết quả mô hình. Trong quá trình thực hiện, nhóm không xem preprocessing là một bước phụ trợ đơn giản, mà xem đây là một thành phần quyết định chất lượng đầu ra. Việc loại bỏ cột trùng lặp, xử lý ký hiệu thiếu, lựa chọn cách mã hóa categorical, chuẩn hóa dữ liệu và xử lý mất cân bằng lớp đều có ảnh hưởng trực tiếp đến hiệu năng cuối cùng.

Nhóm cũng đặc biệt quan tâm đến tác động của các cách biểu diễn dữ liệu khác nhau. Ví dụ, với pipeline truyền thống, nhóm kiểm tra nhánh `full_features` và nhánh giảm chiều bằng PCA 95%. Với pipeline học sâu, nhóm thiết kế các nhánh tiền xử lý khác nhau cho MLP và TabNet. Mục tiêu ở đây là xác định không chỉ mô hình nào tốt, mà còn biểu diễn dữ liệu nào phù hợp với từng mô hình.

1.2.4 Đánh giá tính phù hợp của học sâu trên dữ liệu bảng

Một trong những mục tiêu trung tâm của bài tập lớn là kiểm tra xem học sâu có thực sự tạo ra lợi thế rõ rệt trên bộ dữ liệu bảng như Adult Census Income hay không. Trong nhiều trường hợp, các mô hình truyền thống đã rất mạnh trên dữ liệu dạng này. Vì vậy, nhóm không mặc định rằng mô hình sâu sẽ tốt hơn, mà coi đây là một giả thuyết cần được kiểm chứng bằng thực nghiệm.

Thông qua việc xây dựng MLP, TabNet và các bước tuning tương ứng, nhóm muốn xác định khi nào học sâu giúp cải thiện kết quả, mức cải thiện đó lớn đến đâu, và liệu phần cải thiện có đủ thuyết phục để biện minh cho độ phức tạp bổ sung về mặt huấn luyện và triển khai hay không.

1.2.5 Tăng khả năng diễn giải và tái sử dụng kết quả

Ngoài hiệu năng, nhóm cũng đặt mục tiêu làm cho kết quả có thể giải thích và tái sử dụng. Với pipeline truyền thống, nhóm phân tích độ quan trọng của đặc trưng từ mô hình cuối cùng để liên hệ lại với trực giác dữ liệu. Với toàn bộ dự án, nhóm tổ chức mã nguồn theo các module riêng cho EDA, preprocessing, classical learning, deep learning và tuning để mỗi thành phần có thể tái sử dụng độc lập.

Mục tiêu này giúp bài tập lớn không chỉ dừng ở mức “chạy ra kết quả”, mà tiến gần hơn đến một hệ thống nghiên cứu có cấu trúc rõ ràng, dễ kiểm chứng và có thể phát triển tiếp trong tương lai.

Tóm lại, bài tập lớn hướng tới cả ba lớp mục tiêu: xây dựng đúng quy trình, đánh giá đúng mô hình và rút ra được kết luận có ý nghĩa.

Chương 2

Dữ liệu và phân tích khám phá dữ liệu

2.1 Tổng quan bộ dữ liệu

Trong đề tài này, nhóm sử dụng bộ dữ liệu *Adult Census Income* để xây dựng bài toán phân loại nhị phân mức thu nhập. Mục tiêu là dự đoán một cá nhân thuộc nhóm thu nhập $\leq 50K$ hay $>50K$ dựa trên các thuộc tính nhân khẩu học, học vấn, nghề nghiệp và điều kiện lao động. Đây là một bộ dữ liệu kinh điển trong học máy, đặc biệt phù hợp cho nghiên cứu dữ liệu bảng vì đồng thời chứa biến số, biến phân loại, giá trị thiếu và hiện tượng mất cân bằng lớp.

Xét từ góc độ thực nghiệm, bộ dữ liệu này tạo ra một môi trường so sánh tốt giữa pipeline học máy truyền thống và pipeline học sâu. Dữ liệu đủ lớn để mô hình thể hiện khác biệt rõ ràng, nhưng vẫn nằm trong quy mô có thể khảo sát toàn diện từ khâu làm sạch, phân tích khám phá đến huấn luyện và đánh giá. Ngoài ra, các thuộc tính như độ tuổi, trình độ học vấn, số giờ làm việc mỗi tuần hay nghề nghiệp đều mang ý nghĩa thực tiễn, giúp việc diễn giải kết quả mô hình ở các chương sau trở nên thuyết phục hơn.

2.2 Cấu trúc dữ liệu và các nhóm thuộc tính

Bước đầu tiên nhóm thực hiện là khảo sát cấu trúc tổng quát của dữ liệu, bao gồm số lượng mẫu, số lượng thuộc tính, kiểu dữ liệu của từng cột và thống kê mô tả ban đầu. Mục tiêu của bước này là xác định rõ tập dữ liệu đang gồm những loại biến nào và biến mục tiêu được biểu diễn ra sao.

Qua khảo sát, dữ liệu có thể được chia thành hai nhóm đặc trưng chính:

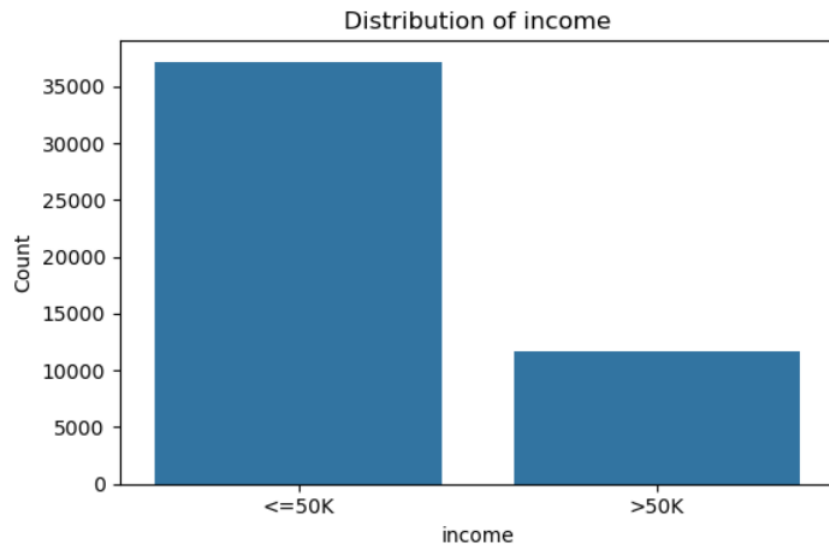
- **Biến số:** gồm các thuộc tính tiêu biểu như `age`, `educational-num`, `hours-per-week`, cùng hai biến tài chính là `capital-gain` và `capital-loss`.
- **Biến phân loại:** gồm các cột tiêu biểu như `workclass`, `occupation`, `relationship`, `race` và `gender`.

Các thuộc tính khác như `native-country` và tình trạng hôn nhân cũng thuộc nhóm này.

Việc nhận diện sớm hai nhóm thuộc tính này có ý nghĩa quan trọng đối với toàn bộ quá trình phân tích, vì cách thống kê, trực quan hóa và xử lý sau đó sẽ khác nhau giữa biến số và biến phân loại. Đồng thời, bước này cũng cho phép nhóm rà soát các cột có nguy cơ trùng lặp thông tin hoặc ít giá trị sử dụng trước khi đi sâu vào mô hình hóa.

2.3 Phân tích khám phá dữ liệu

2.3.1 Phân bố của biến mục tiêu

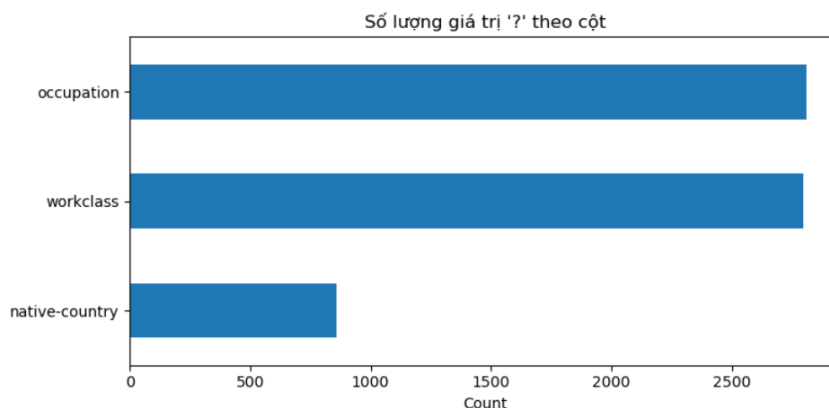


Hình 2.1: Biểu đồ so sánh số lượng 2 lớp của biến mục tiêu

Một trong những nội dung được kiểm tra sớm nhất là phân bố của biến mục tiêu thu nhập. Kết quả cho thấy số lượng mẫu thuộc nhóm $\leq 50K$ lớn hơn đáng kể so với nhóm $>50K$. Điều này cho thấy bài toán có hiện tượng mất cân bằng lớp ở mức vừa phải.

Phát hiện này ảnh hưởng trực tiếp đến cách đánh giá mô hình. Nếu chỉ quan sát độ chính xác tổng thể, mô hình có thể cho kết quả tưởng như cao nhưng thực chất lại dự đoán kém ở lớp thu nhập cao. Vì vậy, ở các chương thực nghiệm sau, nhóm ưu tiên thêm các chỉ số như precision, recall và F1-score để có cái nhìn đầy đủ hơn về hiệu năng phân loại.

2.3.2 Khảo sát giá trị thiếu



Hình 2.2: Số lượng dữ liệu bị thiếu được kí hiệu dấu ?

Trong quá trình khám phá dữ liệu, nhóm nhận thấy một số thuộc tính phân loại xuất hiện ký hiệu ?. Về mặt ngữ nghĩa, đây không phải là một hạng mục hợp lệ mà là biểu diễn của dữ

liệu khuyết thiếu. Các cột như `workclass`, `occupation` và `native-country` là những nơi hiện tượng này xuất hiện rõ nhất.

Kết quả khảo sát giá trị thiếu cho thấy dữ liệu không hoàn toàn sạch và cần có bước chuẩn hóa biểu diễn missing value trước khi đưa vào pipeline học máy. Việc phát hiện sớm đặc điểm này rất quan trọng, vì nếu giữ nguyên ký hiệu ? như một giá trị thông thường thì encoder sẽ xem đó là một nhãn hợp lệ, làm sai lệch không gian đặc trưng và ảnh hưởng đến chất lượng mô hình.

2.3.3 Phân tích các biến số

Đối với nhóm biến số, nhóm xem xét các đại lượng thống kê cơ bản như trung bình, trung vị, độ lệch chuẩn và khoảng giá trị. Một số đặc trưng như `age` và `hours-per-week` có phân bố khá dễ diễn giải, trong khi `capital-gain` và `capital-loss` cho thấy hiện tượng lệch phải mạnh: phần lớn mẫu mang giá trị bằng 0 và chỉ một số ít quan sát có giá trị rất lớn.

Đặc điểm này cho thấy không phải mọi biến số đều có cùng thang đo hoặc cùng hình dạng phân bố. Nếu sử dụng trực tiếp cho các mô hình nhạy với thang đo, các cột có độ phân tán lớn có thể chi phối quá trình học. Đây là một căn cứ quan trọng để ở chương tiếp theo nhóm xem xét các phương án chuẩn hóa dữ liệu số trước khi huấn luyện mô hình.

2.3.4 Phân tích các biến phân loại

Với các thuộc tính phân loại, nhóm khảo sát tần suất xuất hiện của từng nhãn nhằm nhận diện những hạng mục chiếm ưu thế và những hạng mục hiếm. Kết quả cho thấy một số biến như `marital-status`, `relationship` hay `workclass` có phân bố không đồng đều; một vài nhãn xuất hiện rất thường xuyên trong khi một số nhãn khác chiếm tỷ lệ nhỏ.

Thông tin này có giá trị ở hai khía cạnh. Thứ nhất, nó giúp nhóm hiểu rõ bối cảnh xã hội của dữ liệu và những nhóm đối tượng xuất hiện nhiều nhất. Thứ hai, nó cho thấy khi mã hóa các biến phân loại, số chiều đặc trưng có thể tăng lên đáng kể nếu áp dụng one-hot encoding, đặc biệt với các cột có nhiều giá trị phân biệt như `occupation` hoặc `native-country`.

2.3.5 Phát hiện đặc trưng dư thừa

Từ kết quả khảo sát, nhóm nhận thấy cột `education` và `educational-num` phản ánh gần như cùng một nội dung: một cột biểu diễn ở dạng nhãn, cột còn lại biểu diễn ở dạng số thứ bậc. Việc cùng giữ cả hai cột có thể làm mô hình nhận hai phiên bản của cùng một thông tin.

Ngoài ra, cột `fnlwgt` về bản chất là trọng số điều tra dân số, không trực tiếp mô tả đặc điểm cá nhân liên quan đến bài toán dự đoán mức thu nhập. Quan sát này gợi ý rằng không phải mọi cột trong dữ liệu gốc đều nên được đưa nguyên trạng vào mô hình. Nhóm sử dụng kết luận này làm cơ sở cho bước sàng lọc đặc trưng ở chương tiền xử lý.

2.4 Nhận xét rút ra từ giai đoạn EDA

Sau giai đoạn phân tích khám phá, nhóm rút ra bốn nhận xét chính. Thứ nhất, bài toán có mất cân bằng lớp nên cần đánh giá mô hình bằng nhiều chỉ số thay vì chỉ dựa vào accuracy. Thứ hai, dữ liệu tồn tại giá trị thiếu ở một số cột phân loại và cần được chuẩn hóa biểu diễn trước khi mã hóa. Thứ ba, các biến số có thang đo và phân bố khác nhau, đặc biệt có những cột lệch mạnh cần cân nhắc chuẩn hóa phù hợp. Thứ tư, tồn tại các đặc trưng dư thừa hoặc ít ý nghĩa trực tiếp đối với mục tiêu dự đoán.

Mapping between education and educational-num:		
	education	educational-num
0	10th	[6]
1	11th	[7]
2	12th	[8]
3	1st-4th	[2]
4	5th-6th	[3]
5	7th-8th	[4]
6	9th	[5]
7	Assoc-acdm	[12]
8	Assoc-voc	[11]
9	Bachelors	[13]
10	Doctorate	[16]
11	HS-grad	[9]
12	Masters	[14]
13	Preschool	[1]
14	Prof-school	[15]
15	Some-college	[10]

Hình 2.3: Mapping giữa cột education và cột educational-num

Những kết luận này cho thấy giai đoạn EDA không chỉ đóng vai trò mô tả dữ liệu mà còn định hướng trực tiếp cho thiết kế pipeline ở chương tiếp theo. Trên cơ sở đó, **Chương 3** sẽ trình bày chi tiết quá trình tiền xử lý dữ liệu, bao gồm chuẩn hóa missing value, lựa chọn đặc trưng, mã hóa biến phân loại, chuẩn hóa biến số và xây dựng các cấu hình preprocessing dùng trong thực nghiệm.

Chương 3

Tiền xử lý dữ liệu

3.1 Mục tiêu và nguyên tắc thiết kế pipeline

Sau giai đoạn phân tích khám phá dữ liệu ở Chương 2, nhóm xác định rằng chất lượng của mô hình không chỉ phụ thuộc vào thuật toán học máy mà còn phụ thuộc rất lớn vào cách chuẩn bị dữ liệu đầu vào. Vì vậy, chương này tập trung trình bày pipeline tiền xử lý. Toàn bộ pipeline được tổ chức như một nguồn xử lý thống nhất để bảo đảm tính tái lập giữa các thí nghiệm.

Với pipeline học máy truyền thống, mục tiêu của tiền xử lý gồm bốn ý chính: làm sạch dữ liệu, loại bỏ các đặc trưng dư thừa, biến đổi dữ liệu về dạng số phù hợp cho mô hình, và lựa chọn cấu hình preprocessing tốt nhất dựa trên thực nghiệm. Thay vì cố định một lựa chọn ngay từ đầu, nhóm chủ động xây dựng nhiều cấu hình rồi so sánh trên cùng một tập train/test và cùng một mô hình baseline. Cách làm này giúp việc lựa chọn pipeline trở nên có cơ sở định lượng hơn.

Trong suốt quá trình thiết kế, nhóm tuân thủ ba nguyên tắc. Thứ nhất, mọi biến đổi có tham số học được như imputer, encoder hay scaler chỉ được *fit* trên tập huấn luyện để tránh rò rỉ dữ liệu. Thứ hai, việc đánh giá từng cấu hình preprocessing được tách riêng khỏi đánh giá mô hình cuối cùng, nhằm xác định rõ ảnh hưởng của preprocessing. Thứ ba, toàn bộ các thao tác được đóng gói theo dạng hàm hoặc *Pipeline/ColumnTransformer* để dễ tái sử dụng trong các chương tiếp theo.

3.2 Làm sạch dữ liệu ban đầu

3.2.1 Chuẩn hóa biểu diễn giá trị thiếu

Bộ dữ liệu *Adult Census Income* chứa 48,842 mẫu và 15 thuộc tính ban đầu. Trong dữ liệu gốc, giá trị thiếu ở một số cột categorical không được biểu diễn bằng NaN mà bằng ký hiệu ?. Vì vậy, bước đầu tiên của pipeline là chuẩn hóa toàn bộ ký hiệu ? về missing value chuẩn NaN để các bước xử lý sau có thể nhận diện đúng.

Kết quả kiểm tra cho thấy missing value tập trung ở ba cột chính như Bảng 3.1. Các cột còn lại không xuất hiện khuyết thiếu theo cách biểu diễn này.

Việc chuẩn hóa này là nền tảng để các nhánh xử lý sau hoạt động nhất quán. Nếu bỏ qua bước này, bộ mã hóa one-hot sẽ xem ? như một nhãn hợp lệ thay vì dữ liệu thiếu. Khi đó, không gian đặc trưng đầu ra sẽ bị sai lệch ngay từ bước mã hóa.

Bảng 3.1: Số lượng giá trị thiếu sau khi chuẩn hóa ký hiệu ?

Thuộc tính	Số giá trị thiếu
workclass	2,799
occupation	2,809
native-country	857

3.2.2 Loại bỏ đặc trưng dư thừa

Sau bước làm sạch ban đầu, nhóm loại bỏ hai cột `education` và `fnlwgt`. Cột `education` gần như trùng với biến số mô tả trình độ học vấn. Trong khi đó, `fnlwgt` là trọng số điều tra dân số và không phản ánh trực tiếp đặc điểm cần dự đoán của từng cá nhân. Sau bước này, dữ liệu còn lại 13 thuộc tính, gồm 12 đặc trưng đầu vào và 1 biến mục tiêu.

Bước sàng lọc này giúp giảm bớt chiều dữ liệu đầu vào trước khi mã hóa, đồng thời hạn chế nguy cơ để mô hình học từ các tín hiệu kém giá trị hoặc bị lậ thông tin.

3.2.3 Tách đặc trưng và biến mục tiêu

Từ tập dữ liệu đã làm sạch, nhóm tách dữ liệu thành ma trận đặc trưng X và biến mục tiêu y . Kết quả thu được X có kích thước $48,842 \times 12$, còn y gồm 48,842 nhãn thu nhập. Trong đó, lớp $\leq 50K$ có 37,155 mẫu, còn lớp $> 50K$ có 11,687 mẫu. Như vậy, lớp thu nhập cao chiếm xấp xỉ 24% tổng số quan sát.

3.3 Phân nhóm thuộc tính và chia dữ liệu

3.3.1 Xác định biến số và biến phân loại

Sau khi tách X và y , nhóm phân loại 12 đặc trưng đầu vào thành hai nhóm. Nhóm biến số gồm 5 cột: `age`, `educational-num`, `capital-gain`, `capital-loss` và `hours-per-week`. Nhóm biến phân loại gồm 7 cột: `workclass`, `marital-status`, `occupation`, `relationship`, `race`, `gender` và `native-country`.

Việc tách riêng hai nhóm này là điều kiện cần để xây dựng *ColumnTransformer*. Nhờ đó, nhóm có thể áp dụng phép biến đổi phù hợp cho từng kiểu dữ liệu: scaler cho biến số và bộ xử lý missing value kết hợp one-hot encoding cho biến phân loại.

3.3.2 Chia train/test có kiểm soát

Nhóm chia dữ liệu theo tỷ lệ 80/20, tương ứng với 39,073 mẫu train và 9,769 mẫu test. Quá trình chia sử dụng *stratify* theo biến mục tiêu nhằm giữ ổn định phân bố lớp giữa hai tập. Kết quả cho thấy tỷ lệ lớp $> 50K$ ở cả train và test đều xấp xỉ 23.93%, trong khi lớp $\leq 50K$ giữ mức khoảng 76.07%.

Đây là một lựa chọn quan trọng về mặt phương pháp luận. Nếu chia dữ liệu ngẫu nhiên mà không *stratify*, phân bố lớp ở tập test có thể lệch khỏi phân bố gốc, từ đó làm cho việc so sánh các cấu hình preprocessing trở nên kém tin cậy. Ngoài ra, việc chia tập được thực hiện trước mọi bước *fit* của transformer để bảo đảm không có rò rỉ thông tin từ test set vào quá trình tiền xử lý.

```

Train target distribution:
income
<=50K    0.7607
>50K     0.2393
Name: proportion, dtype: float64

Test target distribution:
income
<=50K    0.7607
>50K     0.2393
Name: proportion, dtype: float64

```

Hình 3.1: Kết quả của tập train và test sau khi chia

3.4 Xây dựng các cấu hình preprocessing cho nhánh học máy truyền thống

3.4.1 Thiết kế bằng *Pipeline* và *ColumnTransformer*

Toàn bộ phần tiền xử lý cho nhánh học máy truyền thống được hiện thực bằng *ColumnTransformer*. Trong đó, nhánh numeric nhận trực tiếp danh sách biến số và áp dụng phép chuẩn hóa tương ứng với từng cấu hình. Nhánh categorical được tổ chức dưới dạng *Pipeline* gồm hai bước: *SimpleImputer* để xử lý missing value, sau đó *OneHotEncoder(handle_unknown='ignore')* để mã hóa sang không gian đặc trưng số.

Cách tổ chức này mang lại ba lợi ích. Thứ nhất, mỗi cấu hình preprocessing có thể được tái sử dụng nguyên vẹn ở các bước huấn luyện khác nhau. Thứ hai, mọi phép biến đổi được gắn chặt với đúng nhóm cột cần xử lý, tránh sai sót thủ công. Thứ ba, việc thay đổi một thành phần trong pipeline, chẳng hạn đổi scaler hoặc chiến lược điền giá trị thiếu, trở nên đơn giản và có hệ thống.

3.4.2 Các cấu hình

Nhóm xây dựng bốn cấu hình preprocessing như Bảng 3.2. Sự khác biệt chính nằm ở cách xử lý missing value của nhóm phân loại và cách chuẩn hóa các biến số.

Bảng 3.2: Các cấu hình preprocessing được nhóm so sánh

Mã	Xử lý missing categorical	Xử lý biến số
C1	Điền giá trị xuất hiện nhiều nhất	StandardScaler
C2	Điền hằng Missing	StandardScaler
C3	Điền giá trị xuất hiện nhiều nhất	MinMaxScaler
C4	Điền giá trị xuất hiện nhiều nhất	Không chuẩn hóa

Cả bốn cấu hình đều dùng one-hot encoding cho biến phân loại. Điểm khác biệt chủ yếu nằm ở giả định đối với dữ liệu thiếu và ảnh hưởng của thang đo lên các mô hình tuyến tính.

Đặc biệt, cấu hình C2 xem missing value như một tín hiệu có ý nghĩa riêng bằng cách tạo nhãn Missing, thay vì ép nó về nhóm phổ biến nhất.

3.4.3 So sánh số chiều đầu ra

Do one-hot encoding phụ thuộc vào số lượng nhãn quan sát được ở từng cột, các cấu hình preprocessing có thể sinh ra số chiều đầu ra khác nhau. Kết quả thực nghiệm cho thấy ba cấu hình dùng chiến lược *most frequent* tạo ra 88 đặc trưng sau tiền xử lý. Riêng cấu hình `config_2_onehot_constant_standard` tạo ra 91 đặc trưng, do việc thêm nhãn Missing ở các cột có giá trị khuyết thiếu tạo ra thêm các chiều mới trong không gian one-hot.

Sự khác biệt này rất quan trọng. Nó cho thấy lựa chọn cách điền missing value không chỉ ảnh hưởng đến ý nghĩa thống kê của dữ liệu mà còn ảnh hưởng trực tiếp đến hình dạng không gian đặc trưng mà mô hình học máy quan sát.

3.5 Đánh giá và lựa chọn cấu hình tốt nhất

3.5.1 Chiến lược đánh giá

Để cô lập ảnh hưởng của preprocessing, nhóm sử dụng cùng một mô hình *Logistic Regression* làm bộ đánh giá chuẩn cho cả bốn cấu hình. Với mỗi cấu hình, transformer được *fit* trên X_{train} , sau đó áp dụng lên X_{test} . Mô hình Logistic Regression được huấn luyện với `max_iter = 5000`, `solver = 'lbfgs'` và `random_state = 42`.

Nhóm sử dụng bốn chỉ số đánh giá gồm accuracy, precision, recall và F1-score. Trong đó, F1-score được xem là tiêu chí lựa chọn chính, vì đây là bài toán có mất cân bằng lớp và mục tiêu là đạt được sự cân bằng hợp lý giữa precision và recall trên lớp thu nhập cao >50K.

3.5.2 Kết quả so sánh các cấu hình

Kết quả thực nghiệm được trình bày trong Bảng 3.3. Bảng này cho thấy sự khác biệt giữa các cấu hình không quá lớn về accuracy, nhưng có chênh lệch đáng kể hơn khi quan sát F1-score.

Bảng 3.3: Kết quả so sánh các cấu hình preprocessing bằng Logistic Regression

Cấu hình	Accuracy	Precision	Recall	F1-score	Số đặc trưng
<code>config_2_onehot_constant_standard</code>	0.8554	0.7461	0.5997	0.6649	91
<code>config_4_onehot_mostfreq_noscale</code>	0.8525	0.7387	0.5937	0.6583	88
<code>config_1_onehot_mostfreq_standard</code>	0.8522	0.7375	0.5937	0.6578	88
<code>config_3_onehot_mostfreq_minmax</code>	0.8526	0.7440	0.5855	0.6553	88

Cấu hình `config_2_onehot_constant_standard` đạt F1-score cao nhất là 0.6649, đồng thời cũng cho accuracy cao nhất trong bốn cấu hình. Kết quả này cho thấy việc giữ lại thông tin về missing value dưới dạng một nhãn riêng Missing là có ích hơn so với việc thay thế missing bằng giá trị phổ biến nhất. Nói cách khác, ở bộ dữ liệu Adult, bản thân trạng thái khuyết thiếu của một số thuộc tính categorical cũng có thể mang tín hiệu phân biệt đối với bài toán dự đoán thu nhập.

Ngoài ra, kết quả cũng cho thấy StandardScaler hoạt động ổn định hơn so với MinMaxScaler hoặc việc bỏ qua bước chuẩn hóa khi đi cùng Logistic Regression. Điều này phù hợp với đặc điểm của mô hình tuyến tính, vốn thường hưởng lợi từ dữ liệu số đã được đưa về thang đo chuẩn.

3.5.3 Lựa chọn đầu ra chuẩn cho các chương tiếp theo

Sau bước so sánh, nhóm chọn `config_2_onehot_constant_standard` làm pipeline pre-processing chuẩn cho nhánh học máy truyền thống. Khi *fit* lại cấu hình này trên toàn bộ tập huấn luyện, dữ liệu đầu ra có kích thước $39,073 \times 91$ đối với train và $9,769 \times 91$ đối với test.

Không gian đặc trưng 91 chiều này chính là đầu vào `full_features` được sử dụng ở Chương 4. Từ đầu ra này, nhóm tiếp tục xây dựng thêm nhánh giảm chiều bằng PCA để so sánh giữa việc giữ nguyên toàn bộ thông tin và việc nén đặc trưng trước khi huấn luyện mô hình.

3.6 Kết luận chương

Chương này đã trình bày toàn bộ quy trình tiền xử lý dữ liệu cho nhánh học máy truyền thống theo một pipeline có cấu trúc rõ ràng. Quy trình gồm các bước: chuẩn hóa missing value, loại bỏ cột dư thừa, tách biến đầu vào và biến mục tiêu, phân nhóm thuộc tính, chia train/test có *stratify*, xây dựng nhiều cấu hình preprocessing và lựa chọn cấu hình tốt nhất bằng thực nghiệm.

Đóng góp quan trọng nhất của chương không chỉ nằm ở việc làm sạch dữ liệu, mà còn ở việc chứng minh rằng lựa chọn preprocessing cần được kiểm chứng bằng số liệu thay vì quyết định theo cảm tính. Trên cơ sở đầu ra 91 đặc trưng của cấu hình tốt nhất, Chương 4 sẽ tiếp tục trình bày quá trình trích xuất đặc trưng, huấn luyện baseline và tối ưu hóa các mô hình học máy truyền thống.

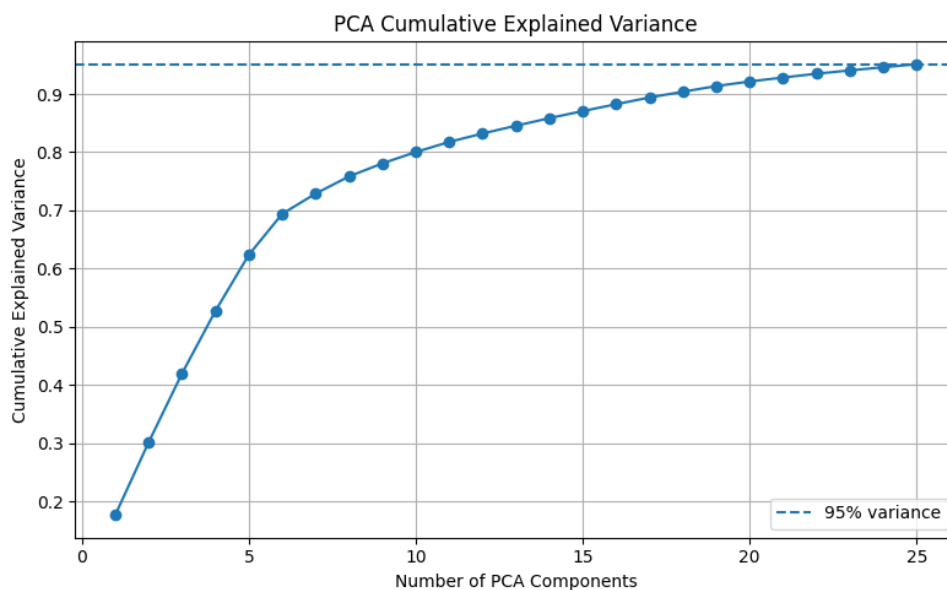
Chương 4

Pipeline học máy truyền thống

4.1 Trích xuất đặc trưng

Dữ liệu sau bước tiền xử lý tạo ra 91 đặc trưng sau khi mã hóa One-hot và chuẩn hóa. Để tối ưu quá trình huấn luyện, nhóm xây dựng hai nhánh đặc trưng:

- Full Features: Giữ nguyên toàn bộ 91 đặc trưng để bảo toàn thông tin chi tiết nhất.
- PCA 95%: Áp dụng phân tích PCA (Principal Component Analysis) trên tập huấn luyện. Kết quả thực nghiệm cho thấy PCA đã nén dữ liệu từ 91 chiều xuống còn 25 thành phần chính mà vẫn giữ lại được 95.12% phương sai của dữ liệu gốc.



Hình 4.1: Biểu đồ phương sai tích lũy của các thành phần chính (PCA)

Biểu đồ phương sai tích lũy (Hình 4.1) cho thấy tốc độ hội tụ thông tin khá nhanh trong những thành phần đầu tiên. Cụ thể, chỉ với khoảng 10 thành phần đầu, mô hình đã giữ lại được hơn 80% phương sai của toàn bộ 91 đặc trưng ban đầu. Việc đạt ngưỡng 95% tại thành phần thứ 25 cho thấy dữ liệu có độ dư thừa nhất định, tạo điều kiện thuận lợi cho việc tối ưu hóa hiệu năng mô hình.

4.2 Huấn luyện mô hình baseline và đánh giá sơ bộ

Sau khi hoàn tất bước trích xuất đặc trưng, nhóm tiến hành thiết lập mức hiệu năng cơ sở (baseline) bằng cách huấn luyện đồng thời các thuật toán học máy truyền thống trên cả hai nhánh đặc trưng: `full_features` và `pca_95`.

4.2.1 Thiết lập huấn luyện Baseline

Nhóm lựa chọn bốn thuật toán có đặc tính toán học khác nhau để đảm bảo tính bao quát:

- **Logistic Regression:** Tính toán nhanh, dễ triển khai và kết quả dễ diễn giải.
- **Linear SVM:** Giúp phân tách dữ liệu trong không gian nhiều chiều. Ưu tiên dùng Linear kernel thay vì RBF để tiết kiệm chi phí tính toán trên tập dữ liệu lớn.
- **Random Forest:** Đại diện cho phương pháp học kết hợp (ensemble), có khả năng xử lý tốt các quan hệ phi tuyến.
- **Gaussian Naive Bayes:** Mô hình xác suất dựa trên giả định độc lập giữa các đặc trưng.

Để giải quyết vấn đề mất cân bằng lớp (lớp thu nhập cao chỉ chiếm khoảng 24% tập dữ liệu), tham số `textttclass_weight='balanced'` được tích hợp vào các mô hình (ngoại trừ Naive Bayes). Chiến lược đánh giá ưu tiên chỉ số F1-score nhằm cân bằng giữa Precision và Recall cho lớp thiểu số.

4.2.2 Kết quả thực nghiệm

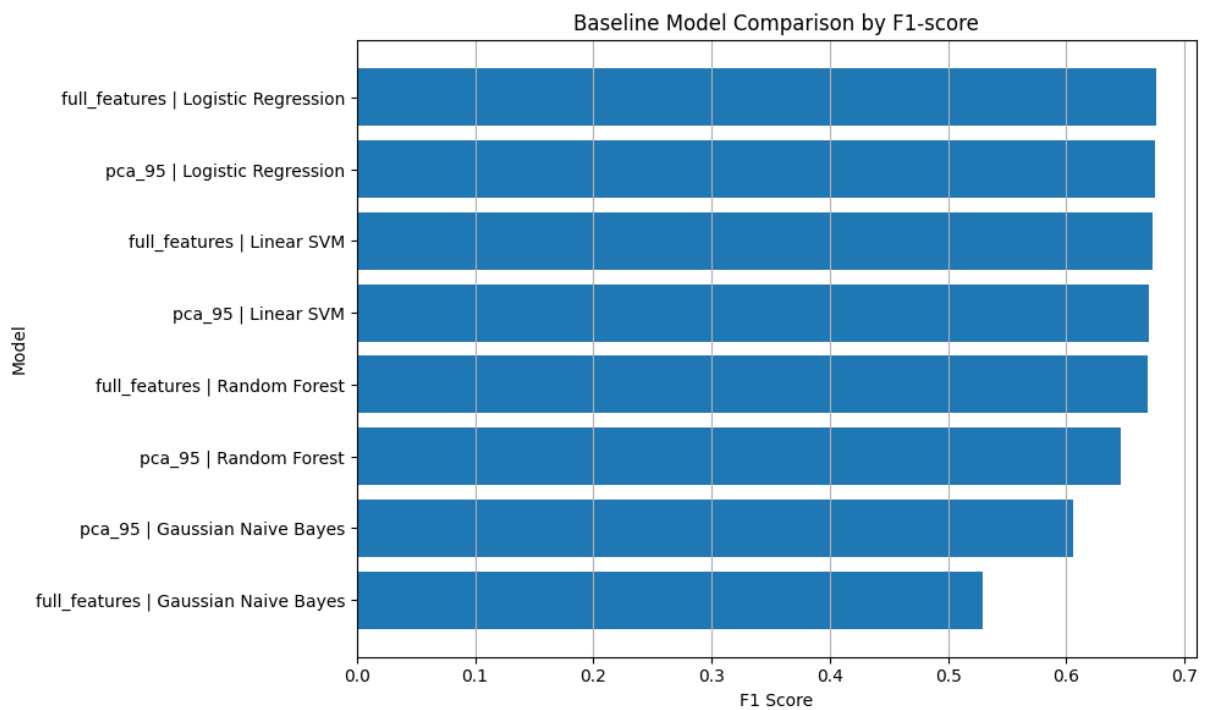
Bảng 4.1: Bảng kết quả huấn luyện Baseline (sắp xếp theo F1-score)

Nhánh đặc trưng	Mô hình	Accuracy	Precision	Recall	F1-score	ROC-AUC
Full Features	Logistic Regression	0.8074	0.5657	0.8396	0.6760	0.9051
PCA 95%	Logistic Regression	0.8057	0.5628	0.8439	0.6752	0.9031
Full Features	Linear SVM	0.8018	0.5561	0.8529	0.6732	0.9045
PCA 95%	Linear SVM	0.7979	0.5500	0.8559	0.6697	0.9025
Full Features	Random Forest	0.8481	0.6993	0.6407	0.6687	0.8940
PCA 95%	Random Forest	0.8396	0.6847	0.6112	0.6459	0.8867
PCA 95%	Gaussian NB	0.8124	0.6093	0.6022	0.6057	0.8402
Full Features	Gaussian NB	0.6014	0.3688	0.9358	0.5291	0.8453

4.2.3 Phân tích và đánh giá sơ bộ

Dựa trên bảng kết quả, nhóm rút ra các nhận định quan trọng sau:

- **Mô hình tốt nhất:** Logistic Regression trên nhánh `full_features` đạt F1-score cao nhất (0.6760). Dù là mô hình tuyến tính đơn giản, nó cho thấy sự ổn định vượt trội trong việc cân bằng giữa Precision và Recall trên không gian đặc trưng One-Hot.



Hình 4.2: Biểu đồ so sánh mô hình Baseline

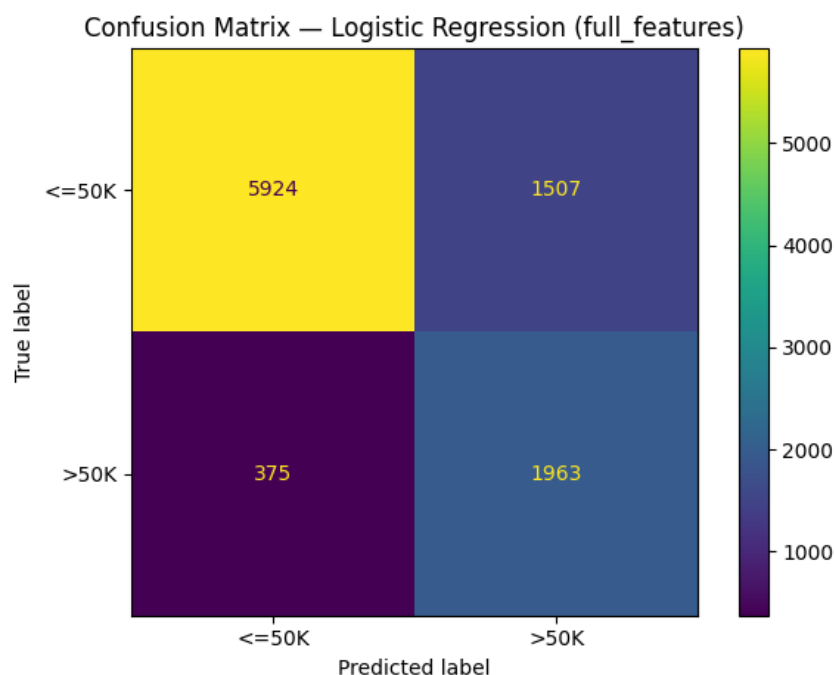
- **PCA:** Việc giảm chiều bằng PCA không giúp cải thiện hiệu năng cho các mô hình mạnh như Logistic Regression hay Random Forest, nhưng lại giúp thời gian huấn luyện giảm đáng kể (VD: Logistic Regression giảm từ 0.51s xuống 0.05s).
- **Naive Bayes:** Mô hình Gaussian Naive Bayes cải thiện rõ rệt F1-score từ 0.5291 lên 0.6057 sau khi áp dụng PCA. Lý do là các thành phần chính (PC) có dạng phân phối liên tục, phù hợp hơn với giả định phân phối Gaussian của mô hình so với dữ liệu mã hóa One-Hot thưa thớt.
- **Mối quan hệ Accuracy - F1:** Random Forest đạt Accuracy cao nhất (0.8481) nhưng F1-score chỉ xếp thứ 5. Điều này minh chứng rằng trong bài toán mất cân bằng lớp, Accuracy có thể gây hiểu lầm nếu không đi kèm với việc phân tích Recall cho lớp thiểu số.

4.2.4 Phân tích chi tiết mô hình Baseline tốt nhất

Nhóm thực hiện phân tích sâu ma trận nhầm lẫn của mô hình Logistic Regression trên nhánh đặc trưng đầy đủ:

- **Lớp $\leq 50K$:** Mô hình dự đoán đúng 5924 mẫu, đạt Recall 0.8.
- **Lớp $> 50K$:** Mô hình đạt Recall 0.84 (đúng 1963/2338 mẫu), nhưng Precision chỉ đạt 0.57, có 1507 mẫu lớp $\leq 50K$ bị dự đoán nhầm là $> 50K$.

Kết luận: Nhánh đặc trưng `full_features` được chọn làm hướng đi chính cho bước tinh chỉnh siêu tham số tiếp theo vì cung cấp đầy đủ thông tin nhất cho các mô hình mạnh.



Hình 4.3: Ma trận nhầm lẫn của mô hình Baseline tốt nhất (Logistic Regression)

4.3 Tinh chỉnh siêu tham số

Dựa trên kết quả đánh giá sơ bộ tại giai đoạn baseline, nhánh đặc trưng `full_features` (91 đặc trưng) cho thấy hiệu năng ổn định và tiềm năng tối ưu hóa cao hơn so với nhánh PCA. Do đó, nhóm lựa chọn nhánh này làm cơ sở để thực hiện quá trình tinh chỉnh siêu tham số nhằm đẩy giới hạn hiệu suất của các mô hình

4.3.1 Chiến lược tối ưu hóa

Để đảm bảo tính khách quan và khả năng tổng quát hóa, quy trình tinh chỉnh được thiết lập dựa trên các nguyên tắc hệ thống:

- **Cross-validation:** Sử dụng phương pháp Stratified 5-Fold Cross Validation. Việc giữ nguyên tỷ lệ lớp trong từng fold giúp mô hình học được ranh giới phân lớp ổn định, đặc biệt quan trọng đối với dữ liệu mất cân bằng.
- **Thước đo tối ưu:** Chỉ số F1-score tiếp tục được chọn làm scoring metric chính để cân bằng giữa Precision và Recall.
- **Phương pháp tìm kiếm:**
 - GridSearchCV: Áp dụng cho các mô hình có không gian tham số nhỏ như Logistic Regression, Linear SVM và Gaussian Naive Bayes để đảm bảo quét toàn bộ các tổ hợp tiềm năng.
 - RandomizedSearchCV: Áp dụng cho Random Forest với `n_iter = 15` nhằm tối ưu hóa thời gian tìm kiếm trên không gian tham số rộng lớn (tổng cộng thực hiện 75 lần huấn luyện).

4.3.2 Kết quả tinh chỉnh chi tiết

Quá trình thực nghiệm đã xác định được bộ tham số tối ưu và các chỉ số hiệu năng tương ứng trên test set:

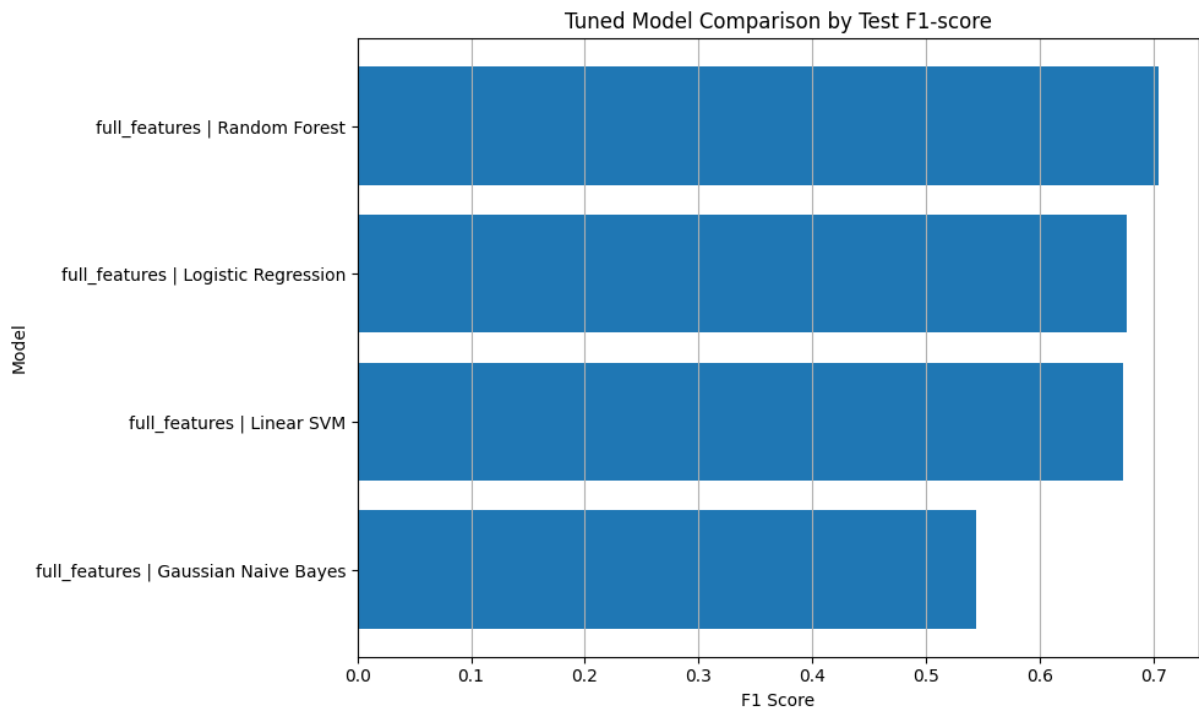
Bảng 4.2: Kết quả tinh chỉnh siêu tham số trên nhánh `full_features`

Mô hình	Tham số tối ưu (Best Parameters)	CV F1	Test F1	ROC-AUC
Random Forest	<code>{'n_estimators': 150, 'min_samples_leaf': 2, ...}</code>	0.7035	0.7050	0.9175
Logistic Regression	<code>{'C': 0.1, 'solver': 'liblinear'}</code>	0.6810	0.6771	0.9052
Linear SVM	<code>{'C': 1.0}</code>	0.6767	0.6732	0.9045
Gaussian NB	<code>{'var_smoothing': 1e-07}</code>	0.5399	0.5438	0.8518

- **Random Forest:** Mô hình đạt mức cải thiện đáng kể nhất. Với cấu hình `n_estimators = 150` và `min_samples_leaf = 2`, mô hình đạt F1-score 0.7050, tăng khoảng 5.4% so với mức baseline (0.6687).
- **Logistic Regression:** Cấu hình `C=0.1` cho thấy việc áp dụng regularization mạnh hơn giúp mô hình ổn định hơn, đạt ROC-AUC 0.9052.
- **Gaussian Naive Bayes:** Việc điều chỉnh `var_smoothing` lên `1e-07` giúp mô hình cải thiện khả năng phân loại trên dữ liệu thưa, đạt F1-score 0.5438 (tăng từ 0.5291).

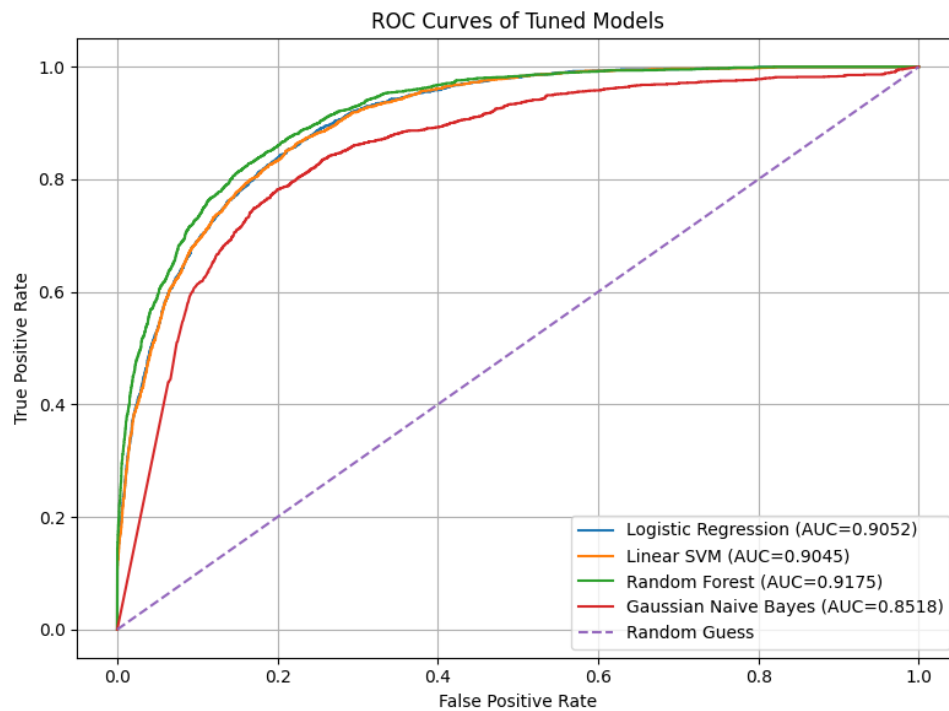
4.3.3 Đánh giá qua trực quan hóa

Hiệu năng của các mô hình sau khi tinh chỉnh được phân tích đa chiều qua các biểu đồ thực nghiệm:



Hình 4.4: So sánh F1-score của các mô hình sau khi tinh chỉnh (Tuned Models)

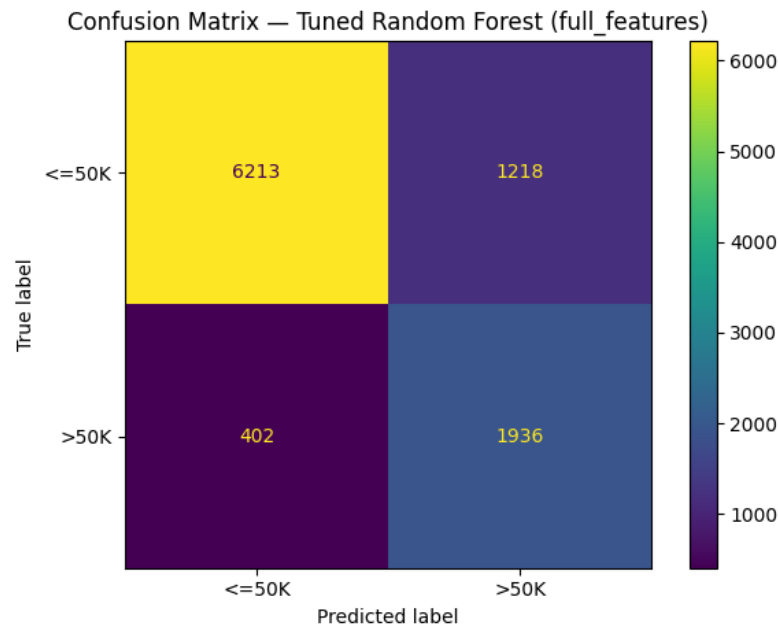
Đường cong ROC (ROC Curves): Hình 4.5 cho thấy khả năng phân tách của các mô hình. Random Forest (AUC=0.9175) chiếm vị trí ưu thế nhất, đường cong tiệm cận góc trên bên trái nhanh hơn các mô hình còn lại, minh chứng cho sự vượt trội trong việc tối ưu hóa cả độ nhạy và độ đặc hiệu.



Hình 4.5: Đường cong ROC của các mô hình sau tinh chỉnh

Ma trận nhầm lẫn (Tuned Random Forest): Qua Hình 4.6, mô hình cuối cùng cho thấy khả năng kiểm soát sai lầm rất cân bằng:

- Dự đoán đúng 6213 mẫu lớp $\leq 50K$ và 1936 mẫu lớp $> 50K$.
- Chỉ số Recall cho lớp thu nhập cao đạt 0.83 và Precision đạt 0.61. So với baseline, mô hình đã giảm bớt được lượng False Positive từ 1507 xuống còn 1218 mẫu, giúp tăng tính tin cậy cho các dự đoán thu nhập cao.



Hình 4.6: Ma trận nhầm lẫn của mô hình Random Forest sau tinh chỉnh

Kết luận: Random Forest trên nhánh `full_features` là thuật toán tối ưu nhất trong pipeline truyền thống, đạt được sự cân bằng lý tưởng giữa các chỉ số phân loại và khả năng chịu lỗi trên dữ liệu thực tế.

4.4 So sánh và lựa chọn mô hình cuối cùng

Sau khi hoàn thành huấn luyện baseline và tinh chỉnh siêu tham số, nhóm tiến hành đối chiếu hiệu năng giữa mô hình cơ sở tốt nhất và mô hình sau tối ưu hóa. Mục tiêu của bước này là xác định cấu hình ưu việt nhất cho pipeline truyền thống dựa trên các chỉ số đánh giá tổng hợp, đặc biệt là F1-score và ROC-AUC.

Bảng 4.3: So sánh hiệu năng giữa mô hình Baseline tốt nhất và mô hình Tuned tốt nhất

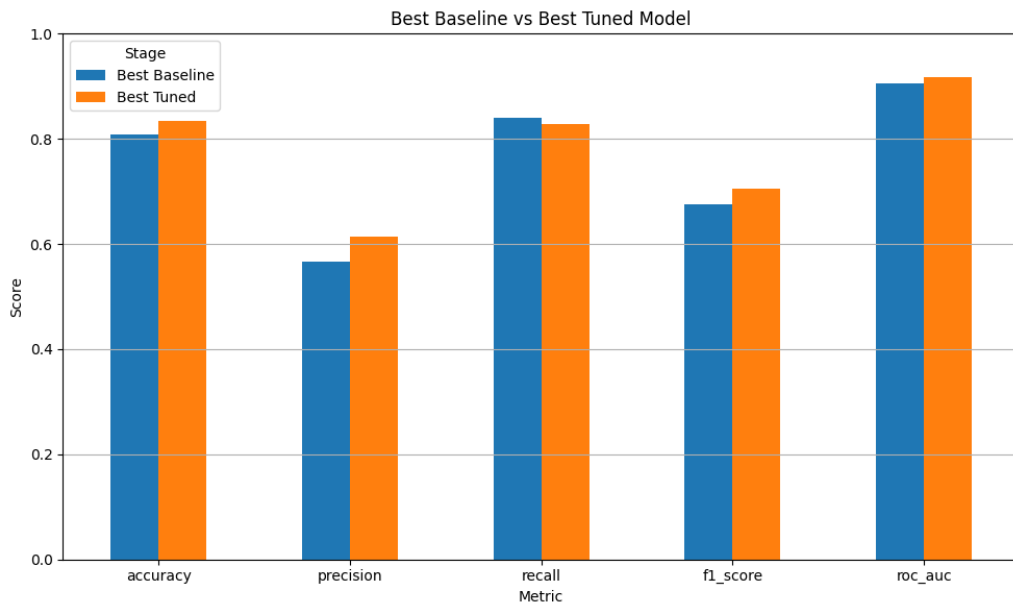
Giai đoạn	Mô hình	Accuracy	Precision	Recall	F1-score	ROC-AUC	Time (s)
Best Baseline	Logistic Regression	0.8073	0.5657	0.8396	0.6759	0.9051	0.51
Best Tuned	Random Forest	0.8341	0.6138	0.8280	0.7050	0.9175	113.85

Kết quả thực nghiệm cho thấy quá trình tinh chỉnh đã giúp cải thiện hiệu năng toàn diện của hệ thống. Cụ thể, mô hình Random Forest sau khi tối ưu hóa cấu hình đã vượt qua Logistic Regression để trở thành mô hình có khả năng phân tách tốt nhất trên tập dữ liệu này.

Bảng 4.4: Mức độ cải thiện các chỉ số chính sau khi tinh chỉnh

Chỉ số	Baseline	Tuned	Delta	Cải thiện (%)
F1-score	0.6760	0.7050	+0.0291	4.30%
Accuracy	0.8073	0.8342	+0.0268	3.32%
ROC-AUC	0.9051	0.9175	+0.0124	1.37%

Mức cải thiện 4.30% ở F1-score và 3.32% ở Accuracy cho thấy việc rà soát kỹ lưỡng không gian siêu tham số đã giúp Random Forest khai thác sâu hơn các quan hệ phi tuyến và tương tác giữa các đặc trưng. Hình 4.7 minh họa trực quan sự chênh lệch này, khẳng định sự vượt trội của cấu hình tinh chỉnh trên hầu hết các chỉ số quan trọng.



Hình 4.7: Biểu đồ so sánh các chỉ số giữa mô hình Baseline và Tuned

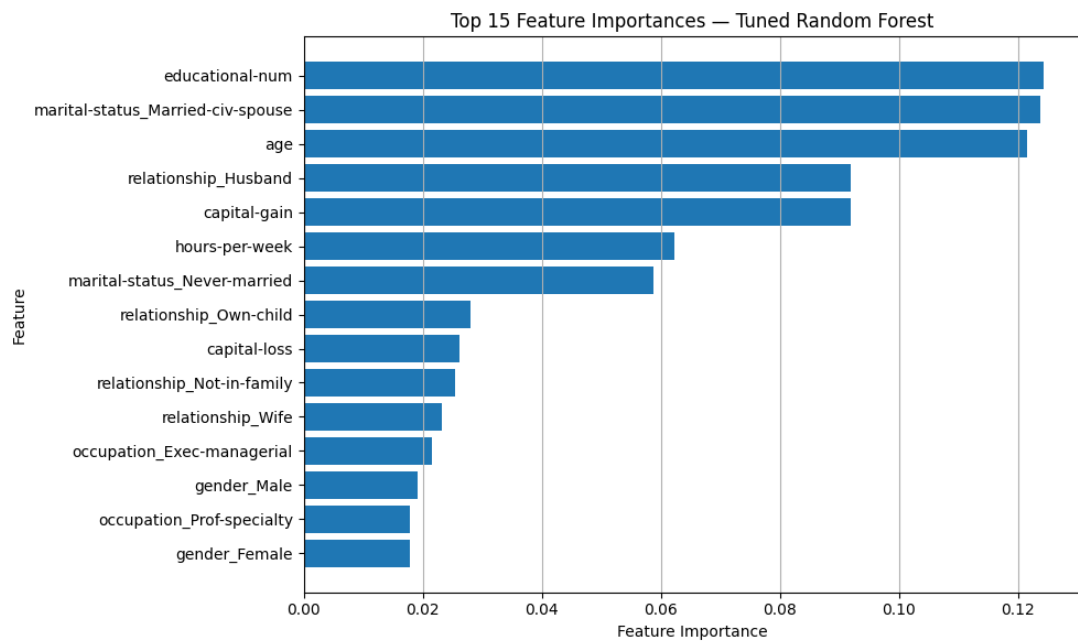
Dựa trên kết quả thực nghiệm, nhóm quyết định lựa chọn mô hình **Tuned Random Forest** làm mô hình cuối cùng cho pipeline truyền thống. Lựa chọn này dựa trên khả năng cân bằng lý tưởng giữa Precision và Recall, đồng thời đạt được diện tích dưới đường cong ROC (AUC) cao nhất, đảm bảo tính tin cậy khi triển khai trên dữ liệu thực tế.

4.5 Phân tích mức độ quan trọng của đặc trưng

Nhằm tăng tính diễn giải cho mô hình, nhóm tiến hành phân tích độ đóng góp của từng đặc trưng vào quyết định của mô hình Random Forest cuối cùng. Bước này giúp kết nối các kết quả định lượng của mô hình với những nhận định định tính đã thực hiện tại giai đoạn EDA.

Biểu đồ mức độ quan trọng (Hình 4.8) xác định các yếu tố mang tính quyết định đến cấu trúc thu nhập trong bộ dữ liệu Adult Census Income. Năm đặc trưng hàng đầu bao gồm:

1. **educational-num (0.1244)**: Số năm học tập là yếu tố hàng đầu, cho thấy trình độ học vấn tỉ lệ thuận với khả năng đạt mức thu nhập cao.
2. **marital-status_Married-civ-spouse (0.1238)**: Tình trạng hôn nhân phản ánh sự ổn định về mặt xã hội thường đi kèm với mức thu nhập ổn định.
3. **age (0.1215)**: Độ tuổi đại diện cho tích lũy về kinh nghiệm làm việc và sự thăng tiến trong sự nghiệp.
4. **relationship_Husband (0.0919)** và **capital-gain (0.0919)**: Cho thấy vai trò trong gia đình và các khoản lợi nhuận tài chính là những chỉ báo mạnh mẽ cho mức thu nhập.



Hình 4.8: Top 15 đặc trưng ảnh hưởng lớn nhất đến dự đoán của mô hình Random Forest

Các đặc trưng khác như số giờ làm việc mỗi tuần (hours-per-week) và các nhóm nghề nghiệp chuyên môn (Exec-managerial, Prof-specialty) cũng đóng góp đáng kể vào khả năng phân tách của mô hình. Kết quả này hoàn toàn tương thích với các quy luật kinh tế - xã hội phổ biến và củng cố thêm tính đúng đắn cho phương pháp tiếp cận của nhóm.

Chương 5

Pipeline học sâu

5.1 Giới thiệu tổng quan

Bên cạnh pipeline học máy truyền thống, nhóm xây dựng thêm một pipeline học sâu để khảo sát khả năng học các quan hệ phi tuyến trên bộ dữ liệu *Adult Census Income*. Mục tiêu của pipeline này không chỉ là triển khai thêm một mô hình khác, mà còn là kiểm chứng xem các kiến trúc học sâu có thực sự mang lại lợi ích rõ rệt đối với dữ liệu bảng trong bài toán dự đoán mức thu nhập hay không.

Nhóm quyết định triển khai hai hướng học sâu chính:

- **MLP (Multi-Layer Perceptron)**: đại diện cho mạng nơ-ron truyền thẳng nhiều lớp áp dụng trên dữ liệu bảng sau khi được chuyển đổi sang dạng số.
- **TabNet**: đại diện cho một kiến trúc học sâu được thiết kế chuyên biệt cho dữ liệu bảng, có cơ chế lựa chọn đặc trưng theo từng bước quyết định.

Ngoài ra, nhóm vẫn giữ một mô hình tuyến tính đơn giản làm mốc tham chiếu trong cùng bối cảnh học sâu. Điều này giúp đánh giá khách quan xem việc chuyển sang mô hình nơ-ron có thực sự tạo ra cải thiện hay chỉ làm tăng độ phức tạp của hệ thống.

5.1.1 Định hướng thiết kế

Nhóm quyết định thiết kế pipeline theo các nội dung sau để đảm bảo công bằng, đúng đắn và cũng như phát huy tối đa khả năng học dữ liệu của các mô hình:

- **Công bằng trong so sánh**: các mô hình được huấn luyện trên cùng bài toán, cùng quy ước nhãn và cùng chỉ số đánh giá bằng Accuracy, Precision, Recall, F1-score và ROC-AUC.
- **Hạn chế rò rỉ dữ liệu**: mọi bước fit của encoder, scaler, imputer hoặc cơ chế cân bằng lớp đều chỉ được thực hiện trên tập huấn luyện.
- **Tối ưu theo đặc thù mô hình**: nhóm không ép mọi mô hình dùng chung một pipeline, mà thiết kế nhánh tiền xử lý riêng sao cho phù hợp với cách học đặc trưng của từng kiến trúc.

5.1.2 Kết quả kỳ vọng

Thông qua việc triển khai pipeline học sâu, nhóm mong muốn trả lời ba câu hỏi thực tế:

1. Liệu MLP có thể vượt qua baseline tuyến tính khi dữ liệu đã được tiền xử lý tốt hay không?
2. Liệu một kiến trúc chuyên biệt cho dữ liệu bảng như TabNet có thể tận dụng cấu trúc dữ liệu tốt hơn MLP hay không?
3. Khi xét cả hiệu năng và độ phức tạp triển khai, pipeline học sâu có đủ thuyết phục để thay thế hoặc bổ sung cho pipeline truyền thống hay không?

Qua đó có được đánh giá giữa các mô hình học sâu và so sánh với các mô hình học máy truyền thống.

5.2 Tiền xử lý dữ liệu học sâu

Nhóm triển khai xây dựng quy trình tiền xử lý biến dữ liệu từ dạng hỗn hợp (gồm biến số, biến phân loại và giá trị khuyết thiếu) thành đầu vào số học nhất quán, ổn định và phù hợp với từng kiến trúc mạng nơ-ron.

5.2.1 Làm sạch dữ liệu và chuẩn hóa đầu vào

Quá trình làm sạch dữ liệu và chuẩn hoá của nhóm trong pipeline này gồm các bước sau:

1. Loại bỏ hai cột `education` và `fnlwgt`. **Lý do:** Cột `education` bị loại vì trùng lặp thông tin với `educational-num`, việc giữ đồng thời cả hai cột sẽ làm tăng tính dư thừa và có thể gây nhiễu cho mô hình. Cột `fnlwgt` bị loại vì đây là trọng số thống kê dùng trong khảo sát dân số, không phản ánh trực tiếp đặc điểm cá nhân cần thiết cho bài toán phân loại thu nhập.
2. Xử lý giá trị thiếu bằng cách chuẩn hóa ký hiệu ? thành giá trị thiếu chuẩn và sau đó loại bỏ hoàn toàn các bản ghi thiếu để giữ tập dữ liệu sạch. **Lý do:** Giúp mô hình giảm nhạy cảm với nhiễu và tránh việc học từ các biểu diễn không nhất quán.
3. Mã hóa biến mục tiêu về dạng nhị phân. Cụ thể, nhóm thu nhập thấp hơn hoặc bằng 50K được gán thành lớp 0, còn nhóm thu nhập lớn hơn 50K được gán thành lớp 1. **Lý do:** Giúp toàn bộ pipeline học sâu sử dụng thống nhất các hàm loss cho bài toán phân loại nhị phân.

Sau quá trình làm sạch và chuẩn hoá thu được:

```
Dropped columns: ['education', 'fnlwgt']
Dropped rows with missing values: 3620
Mapped target variable 'income' to binary (0 < 1).

Kích thước sau làm sạch: (45222, 13)
```

Hình 5.1: Kết quả quá trình làm sạch và chuẩn hoá

5.2.2 Chia tập dữ liệu

Sau khi làm sạch dữ liệu, nhóm chia dữ liệu thành ba phần: train, validation và test. Việc chia ba tập thay vì chỉ hai tập là cần thiết đối với học sâu vì cần một tập validation riêng để theo dõi quá trình học, kích hoạt cơ chế dừng sớm và lựa chọn siêu tham số.

Nhóm sử dụng chiến lược chia có *stratify* theo nhãn để giữ tỷ lệ hai lớp gần như nhất quán giữa các tập.

Lý do: Điều này đặc biệt quan trọng vì bài toán thu nhập có mất cân bằng lớp: số mẫu thuộc lớp thu nhập thấp lớn hơn đáng kể so với lớp thu nhập cao. Nếu không chia có kiểm soát, mô hình có thể được huấn luyện hoặc đánh giá trên các tập có phân phối lệch nhau quá nhiều, dẫn đến kết quả thiếu ổn định.

```
Split → Train: 31654, Val: 4523, Test: 9045
Class dist (train) >50K: 24.8%
```

Hình 5.2: Kết quả chia dữ liệu

Chú ý: Việc tách tập được thực hiện trước bước mã hóa và chuẩn hóa. Nhóm chọn cách này để mọi biến đổi đều chỉ được fit trên dữ liệu huấn luyện, từ đó loại bỏ nguy cơ rò rỉ thông tin từ validation/test vào quá trình học.

5.2.3 Tiền xử lý cho MLP

Đối với MLP, nhóm lựa chọn biểu diễn dữ liệu theo hướng truyền thống: biến phân loại được mã hóa bằng *One-Hot Encoding*, còn biến số được chuẩn hóa bằng *StandardScaler*.

Lý do: MLP làm việc tốt khi đầu vào là các vector số liên tục có cùng thang đo.

Nhóm áp dụng one-hot encoding cho toàn bộ các cột phân loại nhằm chuyển dữ liệu về dạng nhị phân dễ xử lý. Lợi ích của bước này là mô hình có thể tiếp cận từng trạng thái phân loại như những đặc trưng riêng biệt, từ đó học được trọng số tương ứng cho từng nhóm nghề nghiệp, tình trạng hôn nhân, quốc gia, giới tính hoặc quan hệ gia đình. Tuy nhiên, đổi lại số chiều đầu vào tăng lên đáng kể. Nhóm chấp nhận đánh đổi này vì MLP vẫn có thể xử lý tốt trong quy mô dữ liệu của đề tài.

```
X_train_oh, ohe_encoder = apply_oh(X_train_raw, cat_cols)
X_val_oh, _ = apply_oh(X_val_raw, cat_cols, encoder=ohe_encoder)
X_test_oh, _ = apply_oh(X_test_raw, cat_cols, encoder=ohe_encoder)

print(f"\nShape sau OHE → train: {X_train_oh.shape} | val: {X_val_oh.shape} | test: {X_test_oh.shape}")
✓ 0.0s

Applied OHE on 7 columns → 82 new columns.
Applied OHE on 7 columns → 82 new columns.
Applied OHE on 7 columns → 82 new columns.

Shape sau OHE → train: (31654, 87) | val: (4523, 87) | test: (9045, 87)
```

Hình 5.3: Kết quả One-Hot Encoding

Với các đặc trưng số, nhóm dùng chuẩn hóa z-score để đưa các cột về cùng thang đo.

Lý do: Giúp gradient descent hội tụ ổn định hơn, tránh hiện tượng một vài biến có giá trị lớn lấn át những biến còn lại.

```

X_train_sc, scaler = scale_numeric(X_train_ohe, num_cols)
X_val_sc, _ = scale_numeric(X_val_ohe, num_cols, scaler=scaler)
X_test_sc, _ = scale_numeric(X_test_ohe, num_cols, scaler=scaler)
✓ 0.0s
Scaled 5 numeric columns: ['age', 'educational-num', 'capital-gain', 'capital-loss', 'hours-per-week']
Scaled 5 numeric columns: ['age', 'educational-num', 'capital-gain', 'capital-loss', 'hours-per-week']
Scaled 5 numeric columns: ['age', 'educational-num', 'capital-gain', 'capital-loss', 'hours-per-week']

```

Hình 5.4: Kết quả chuẩn hoá về cùng thang đo

5.2.4 Xử lý mất cân bằng lớp

Nếu giữ nguyên phân bố gốc, mô hình có thể nghiêng về dự đoán lớp đa số và đạt accuracy khá cao nhưng lại bỏ sót nhiều mẫu thuộc lớp thu nhập cao.

Nhóm xử lý vấn đề này theo hai cách khác nhau tùy mô hình:

Với **Logistic Regression baseline** trong bối cảnh học sâu, nhóm sử dụng SMOTE trên tập train để sinh thêm mẫu cho lớp thiểu số. Cách làm này giúp tạo ra một mốc tham chiếu mạnh hơn cho baseline và kiểm tra xem việc cân bằng dữ liệu đầu vào có cải thiện khả năng phân loại hay không.

Tuy nhiên, với **MLP**, nhóm không sử dụng SMOTE sau one-hot encoding làm pipeline chính.

Lý do: SMOTE nội suy tuyến tính giữa các mẫu, trong khi các cột one-hot chỉ có ý nghĩa ở mức 0 hoặc 1. Nếu nội suy trên không gian này, mô hình sẽ nhận được những giá trị trung gian không còn mang ý nghĩa phân loại rõ ràng. Thay vì vậy, nhóm xử lý mất cân bằng lớp trực tiếp trong hàm loss bằng `pos_weight`. Cách này giúp mô hình chú ý hơn đến lớp thiểu số mà không làm biến dạng không gian đặc trưng.

5.2.5 Tiền xử lý riêng cho TabNet

TabNet được thiết kế chuyên biệt cho dữ liệu bảng và có khả năng xử lý biến phân loại theo cách hiệu quả hơn MLP nên nhóm quyết định thiết kế quá trình tiền xử lý riêng. Cụ thể các bước:

1. Thay vì one-hot encoding, nhóm mã hóa các biến phân loại thành các chỉ số nguyên.

Lý do: Cách biểu diễn này cho phép TabNet sử dụng embedding nội bộ, qua đó học được mối quan hệ giữa các giá trị phân loại mà không tăng quá nhiều số chiều đầu vào.

2. Gom các nhóm giá trị quá hiếm về một nhóm chung.

Lý do: Giúp giảm nhiễu, tránh để mô hình phải học từ những trạng thái xuất hiện rất ít nhưng vẫn chiếm chỗ trong không gian biểu diễn.

3. Áp dụng quantile binning cho một số biến liên tục như tuổi và số giờ làm việc mỗi tuần.

Mục đích: Chuyển các biến có phân phối lệch thành những khoảng mang ý nghĩa rời rạc hơn, qua đó phù hợp hơn với cơ chế xử lý dạng categorical của TabNet.

5.3 Mạng nơ-ron học đặc trưng

5.3.1 MLP – Multi-Layer Perceptron

MLP được nhóm xem như mô hình học sâu cơ bản nhất để kiểm tra sức mạnh của một mạng nơ-ron truyền thẳng khi đầu vào đã được tiền xử lý kỹ. Về cấu trúc, nhóm xây dựng MLP theo mô hình nhiều tầng ẩn, trong đó mỗi tầng bao gồm phép biến đổi tuyến tính, chuẩn hóa batch, hàm kích hoạt ReLU và dropout.

Nhóm chọn cấu trúc này vì mỗi thành phần đều có vai trò rõ ràng:

- Lớp tuyến tính giúp mô hình học tổ hợp trọng số trên các đặc trưng đầu vào,
- Batch normalization giúp phân phối kích hoạt ổn định hơn giữa các epoch,
- ReLU đưa vào tính phi tuyến,
- Dropout giúp giảm nguy cơ overfitting.

Trong thí nghiệm cơ sở, nhóm dùng kiến trúc ba tầng ẩn với kích thước giảm dần. Cách tổ chức này giúp mô hình học được các biểu diễn trung gian từ tổng quát đến cô đọng hơn.

Lưu ý: Nhóm không chọn mạng quá sâu vì dữ liệu của bài toán là dữ liệu bảng với số mẫu hữu hạn; nếu dùng kiến trúc quá lớn, mô hình rất dễ học thuộc dữ liệu train nhưng không cải thiện tốt trên validation.

5.3.2 TabNet – kiến trúc chuyên biệt cho dữ liệu bảng

Khác với MLP, TabNet được thiết kế đặc biệt cho dữ liệu dạng bảng. Điểm nổi bật của TabNet là mô hình không nhìn toàn bộ đặc trưng theo cùng cách ở mọi bước, mà dùng cơ chế attention để chọn ra các đặc trưng quan trọng tại từng giai đoạn ra quyết định.

Nhóm quyết định chọn TabNet vì đây là một kiến trúc có tính hiện đại, đồng thời phù hợp về mặt lý thuyết với dữ liệu Adult Census Income. Khi biến phân loại được mã hóa thành chỉ số nguyên và đưa qua embedding nội bộ, TabNet có khả năng học ra các biểu diễn tinh gọn hơn so với one-hot encoding thuần túy. Ngoài ra, việc chọn lọc đặc trưng theo từng bước còn mang lại lợi thế diễn giải tốt hơn so với MLP thông thường.

Lưu ý: TabNet có nhiều siêu tham số và nhạy với cách chia dữ liệu, batch size, cơ chế pretraining cũng như cách mã hóa đầu vào, có thể dẫn đến kết quả không như mong đợi. Vì vậy, TabNet được nhóm dùng để kiểm định một kiến trúc nâng cao, chứ không phải là lựa chọn mặc nhiên thay thế cho MLP.

5.4 Huấn luyện đầu-cuối

5.4.1 Thiết lập thí nghiệm

Trước khi huấn luyện, notebook đặt seed cho random, numpy và torch với giá trị 42. Nếu GPU khả dụng, code cũng tắt một số tối ưu ngẫu nhiên của cuDNN để đảm bảo tính xác định nhiều nhất có thể.

Mục đích: Giảm sai khác kết quả giữa các lần chạy.

Thiết bị huấn luyện được chọn tự động qua biến device. Cụ thể, code ưu tiên 'cuda' khi `torch.cuda.is_available()` trả về đúng, và ngược lại sẽ dùng 'cpu'.

Mục đích: cùng một notebook có thể chạy trên máy cá nhân hoặc Google Colab.

5.4.2 Huấn luyện MLP cơ bản

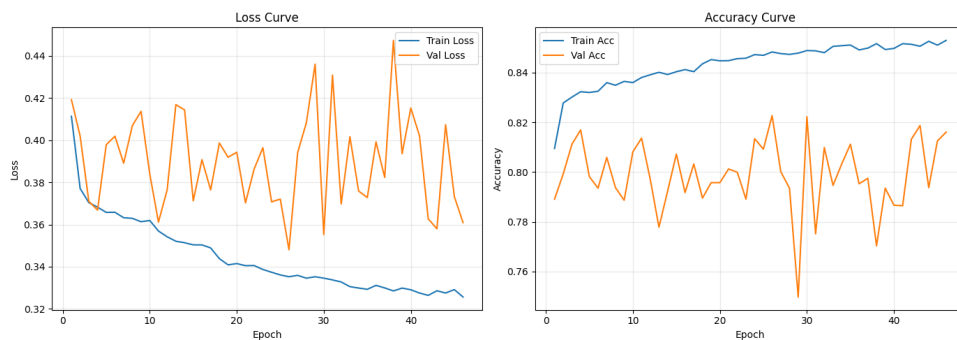
Nhóm huấn luyện MLP cơ bản bằng hàm `train_model()`. Vòng huấn luyện gồm các thành phần chính sau:

- **Loss:** `BCEWithLogitsLoss`, có thể dùng `pos_weight` để bù mất cân bằng lớp.
- **Optimizer:** Adam.
- **Scheduler:** `ReduceLROnPlateau` với hệ số giảm 0.5 và `patience=5`.
- **Early stopping:** dừng khi `val_loss` không còn cải thiện sau một số epoch liên tiếp.

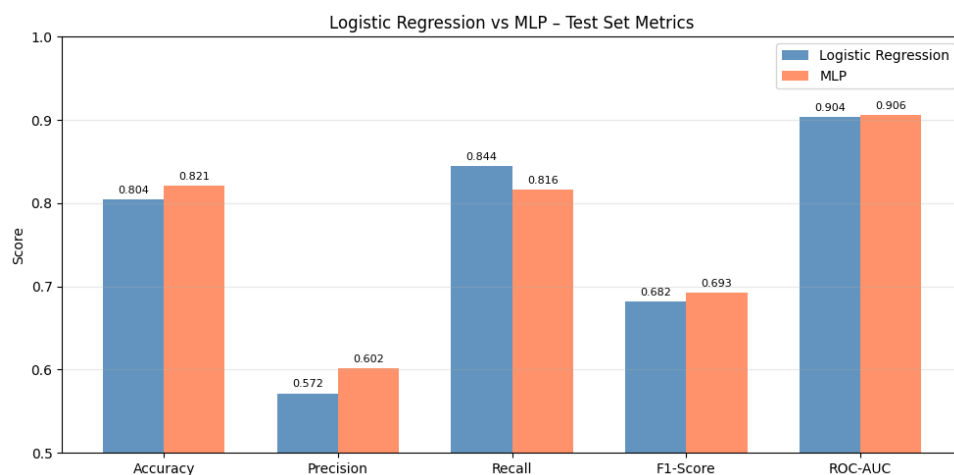
Cấu hình cơ bản cho MLP mà nhóm đưa ra ban đầu là:

epochs	lr	weight_decay	patience	use_pos_weight
100	1e-3	1e-3	20	True

Lịch sử huấn luyện được lưu trong `history` gồm `train_loss`, `val_loss`, `train_acc`, `val_acc`; sau đó hàm `plot_learning_curves()` được dùng để trực quan hóa đường cong học tập.



Hình 5.5: Đường cong học tập với các tham số cơ bản



Hình 5.6: So sánh kết quả Test set giữa Logistic Regression và MLP

Nhận xét: Model học tốt trên tập train nhưng không cải thiện trên validation, cho thấy dấu hiệu overfitting. Đồng thời khi so sánh với Logistic Regression có cải thiện nhẹ, nhóm sẽ tiếp tục triển khai tìm tham số tối ưu với Optuna.

5.4.3 Tối ưu MLP bằng Optuna

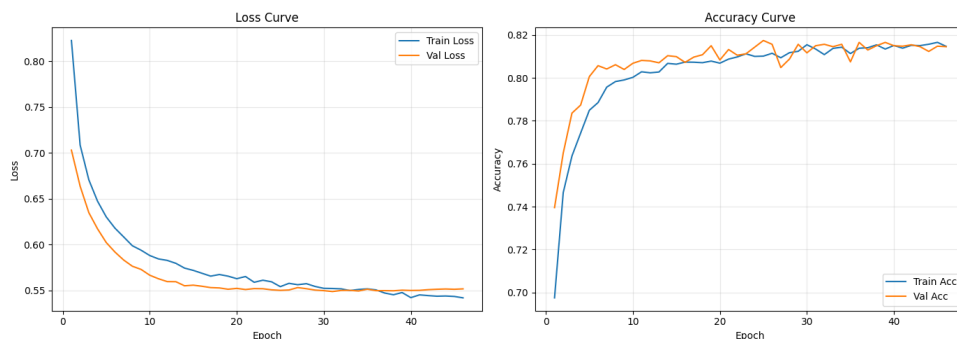
Mã trong `modules/tuning.py` sử dụng Optuna để tìm siêu tham số tốt hơn cho MLP. Không gian tìm kiếm bao gồm:

- số hidden layer `n_layers`,
- kích thước từng hidden layer `hidden_dim_i`,
- dropout,
- learning rate,
- weight decay,
- batch size,
- lựa chọn optimizer giữa Adam và AdamW.

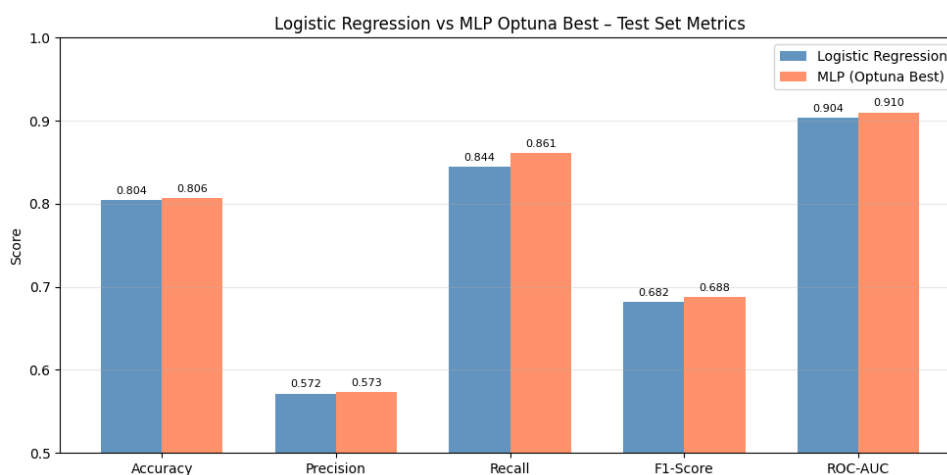
Optuna dùng `TPESampler(seed=42)` kết hợp `MedianPruner` để loại bỏ sớm các trial kém hiệu quả. Trong lần chạy gần nhất, bộ tham số tốt nhất cho MLP được ghi nhận là:

<code>n_layers</code>	3
<code>hidden_dims</code>	[320, 448, 32]
<code>dropout</code>	0.35
<code>learning_rate</code>	$\approx 1.52 \times 10^{-4}$
<code>weight_decay</code>	$\approx 3.03 \times 10^{-5}$
<code>batch_size</code>	512
<code>optimizer</code>	AdamW

Sau khi có cấu hình tốt nhất, hàm `train_best_optuna_model()` được dùng để huấn luyện lại mô hình trên pipeline dữ liệu đã chuẩn bị và đánh giá trên tập test.



Hình 5.7: Đường cong học tập với các tham số sau Optuna



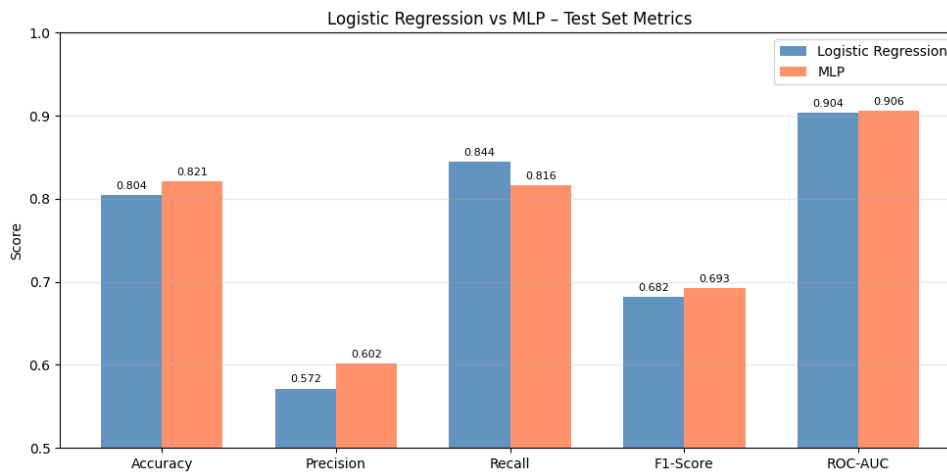
Hình 5.8: So sánh kết quả Test set giữa Logistic Regression và MLP (Optuna)

Nhận xét: Model với các siêu tham số mới này đã thể hiện việc học có cải thiện tốt hơn trên tập dữ liệu validation so với các tham số cơ bản ban đầu nhóm đề ra. Đồng thời các chỉ số cũng đã cải thiện về mọi mặt so với Logistic Regression.

5.4.4 Huấn luyện TabNet và pretraining

TabNet được huấn luyện bằng hàm `train_tabnet_model()`. Cấu hình cơ bản ban đầu của TabNet mà nhóm sử dụng:

n_d = n_a	32
n_steps	3
gamma	1.3
learning_rate	2e-2
max_epochs	100
patience	15
batch_size	512
virtual_batch_size	128



Hình 5.9: So sánh giữa Logistic Regression, MLP(Optuna) và TabNet

Nhận xét: TabNet với các tham số mà nhóm đưa ra chưa cải thiện được so với MLP, thậm chí còn thấp hơn ở F1-Score. Do đó, nhóm tiếp tục tối ưu siêu tham số của TabNet với Optuna.

Sau khi huấn luyện mô hình cơ bản, nhóm tiếp tục dùng Optuna để tối ưu TabNet và kết hợp thêm bước **self-supervised pretraining** thông qua hàm `pretrain_tabnet()`. Ở bước này, mô hình học tái tạo dữ liệu đầu vào bị che một phần nhằm tạo ra biểu diễn khởi đầu tốt hơn trước khi huấn luyện có giám sát.

width	8
n_steps	4
gamma	≈ 1.0138
lambda_sparse	≈ 0.003924
learning_rate	≈ 0.007085
batch_size	512
virtual_batch_size	256
pretrain_epochs	50
pretraining_ratio	≈ 0.6195

Đây là kết quả cấu hình tốt nhất của TabNet mà Optuna tìm được. Sau được dùng để so sánh đánh giá với các mô hình hiện có.

5.5 Đánh giá

Sau khi hoàn tất quá trình huấn luyện, nhóm tiến hành đánh giá từng mô hình trên tập test độc lập. Nhóm sử dụng đồng thời nhiều chỉ số, bao gồm Accuracy, Precision, Recall, F1-score và ROC-AUC, thay vì chỉ dựa vào một chỉ số duy nhất.

5.5.1 Phương pháp đánh giá

Nhóm lựa chọn F1-score là chỉ số trung tâm khi so sánh các mô hình.

Lý do: Bài toán dự đoán mức thu nhập có mất cân bằng lớp: nếu chỉ xét accuracy, một mô hình thiên về lớp đa số vẫn có thể đạt kết quả cao nhưng lại không phát hiện tốt các mẫu thuộc

lớp thu nhập cao. F1-score giúp cân bằng giữa precision và recall, qua đó phản ánh tốt hơn chất lượng thực sự của mô hình.

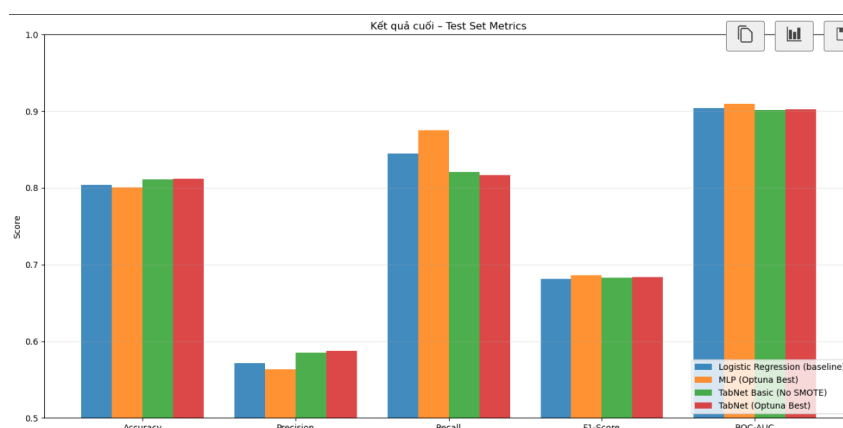
Bên cạnh đó, nhóm vẫn theo dõi ROC-AUC để đánh giá khả năng tách lớp trên nhiều ngưỡng khác nhau. Điều này đặc biệt hữu ích khi so sánh các mô hình có hành vi dự đoán khác nhau nhưng cùng đạt mức accuracy tương đối gần nhau.

Ngoài các chỉ số tổng hợp, nhóm còn dùng *classification report* và *confusion matrix*. Hai công cụ này giúp nhìn rõ hơn các dạng sai sót mà mô hình mắc phải, chẳng hạn mô hình có xu hướng bỏ sót lớp thu nhập cao hay lại dự đoán nhầm quá nhiều mẫu thu nhập thấp thành thu nhập cao.

5.5.2 Kết quả thực nghiệm

Mô hình	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression (baseline)	0.8043	0.5712	0.8448	0.6815	0.9038
MLP (Optuna Best)	0.8011	0.5636	0.8756	0.6858	0.9099
TabNet Basic	0.8112	0.5848	0.8211	0.6831	0.9020
TabNet (Optuna Best)	0.8124	0.5874	0.8171	0.6835	0.9029

Bảng 5.1: So sánh kết quả cuối cùng trên tập test.



Hình 5.10: Hình minh họa kết quả tổng hợp của pipeline học sâu.

Qua thực nghiệm, nhóm nhận thấy rằng Logistic Regression baseline vẫn là một đối thủ rất mạnh. Điều này cho thấy với dữ liệu bảng đã được tiền xử lý hợp lý, các mô hình tuyến tính hoặc bán tuyến tính vẫn có thể đạt hiệu năng cao và ổn định.

MLP cơ bản cho thấy mô hình có khả năng học được trên dữ liệu, nhưng chưa tạo ra khoảng cách đủ lớn so với baseline. Khi nhóm tối ưu MLP bằng Optuna, kết quả được cải thiện hơn, chứng tỏ phần tuning là cần thiết nếu muốn khai thác tốt hơn khả năng của mạng nơ-ron.

Đối với TabNet, nhóm ghi nhận rằng đây là một mô hình có tiềm năng về mặt kiến trúc, nhưng kết quả thực nghiệm không tự động vượt trội chỉ vì mô hình phức tạp hơn. Ở cấu hình cơ sở, TabNet chưa vượt được MLP và baseline. Vì vậy, nhóm tiếp tục tối ưu bằng Optuna và dùng thêm pretraining để kiểm tra liệu lợi thế kiến trúc của TabNet có thể được khai thác tốt hơn hay không. Kết quả không cải thiện lắm so với mong đợi. Điều này có thể đến từ thiếu sót trong quá trình huấn luyện của nhóm.

Kết luận: Học sâu có thể cải thiện hiệu năng trên dữ liệu bảng khi được tiền xử lý phù hợp. Optuna cũng giúp cải thiện chất lượng mô hình. Tuy nhiên so với baseline truyền thống thì học sâu không phải lúc nào cũng vượt trội hơn.

5.5.3 Triển khai mô hình

Sau khi thực hiện thí nghiệm với các mô hình học sâu và đưa ra so sánh đánh giá. Dựa trên mô hình học sâu tốt nhất, nhóm tiến hành triển khai một ứng dụng thực tế bằng thư viện Streamlit.

Mô hình học sâu tốt nhất sẽ được chọn dựa trên chỉ số quan trọng đối với dữ liệu mất cân bằng là F1-Score. Sau khi chọn ra mô hình, hệ thống sẽ lưu lại loại mô hình, các bộ tiền xử lý, tham số, cấu hình của mô hình đó vào folder models.

Hướng dẫn chạy ứng dụng:

1. Tại thư mục của dự án, mở terminal và gõ lệnh `streamlit run app.py`
2. Nếu có các thư viện chưa cài đặt, cần phải cài đặt trước khi chạy lệnh
3. Nếu xuất hiện thông báo thiếu file cấu hình, nguyên nhân là cấu hình mô hình chưa được lưu. Người dùng cần chạy xong notebook `modules/04_deep_learning.ipynb` để tạo các tệp cần thiết.
4. Ứng dụng hiển thị và người dùng nhập các thông tin bao gồm: thông tin cá nhân (tuổi, giới tính, chủng tộc, quốc gia, tình trạng kết hôn, vị trí trong gia đình), thông tin việc làm và giáo dục (tầng lớp lao động, trình độ giáo dục, nghề nghiệp), thông tin tài chính (lợi nhuận vốn, lỗ vốn (theo đồng Đôla) và số giờ làm việc mỗi tuần)
5. Nhấn **PREDICT INCOME** và ứng dụng sẽ trả ra kết quả dự đoán thu nhập/năm.

Sau đây là giao diện cơ bản của ứng dụng:

Adult Income Prediction App

Fill in or select the information below to let the deep learning model infer whether the income exceeds \$50K/year.

Personal & Demographic Information

Age
39

Gender
Male

Race
White

Native Country
United States

Marital Status
Married (Civilian Spouse)

Relationship
Husband

PREDICT INCOME

Work & Education Information

Work Class
Private

Educational Num
10

Occupation
Professional / Specialty

Financial Information

Capital Gain
1101

Capital Loss
89

Hours per Week
1

Hình 5.11: Giao diện chính của ứng dụng

Personal & Demographic Information

Age
39

Gender
Male

Race
White

Native Country
United States

Marital Status
Married (Civilian Spouse)

Relationship
Husband

PREDICT INCOME

Work & Education Information

Work Class
Private

Educational Num
10

Occupation
Professional / Specialty

Financial Information

Capital Gain
1101

Capital Loss
89

Hours per Week
1

PREDICTION RESULT

Your Predicted Income Per Year Is

High Income Bracket ($\geq$ \$50K)

Hình 5.12: Kết quả dự đoán thu nhập $\geq$ 50K

Personal & Demographic Information

Age: 39

Gender: Male

Race: White

Occupation: Professional / Specialty

Native Country: United States

Marital Status: Married (Civilian Spouse)

Relationship: Husband

Financial Information

Capital Gain: 1101

Capital Loss: 89

Hours per Week: 1

PREDICTION RESULT

Your Predicted Income Per Year Is

Standard Income Bracket (< \$50K)

PREDICT INCOME

Hình 5.13: Kết quả dự đoán thu nhập <50K

Chương 6

So sánh kết quả và phân tích

6.1 So sánh giữa pipeline truyền thống và pipeline học sâu

Sau khi hoàn thành quá trình huấn luyện và đánh giá các mô hình trong hai pipeline, nhóm tiến hành tổng hợp toàn bộ kết quả nhằm so sánh hiệu năng giữa pipeline học máy truyền thống và pipeline học sâu trên bộ dữ liệu Adult Census Income.

Các mô hình được đưa ra so sánh bao gồm:

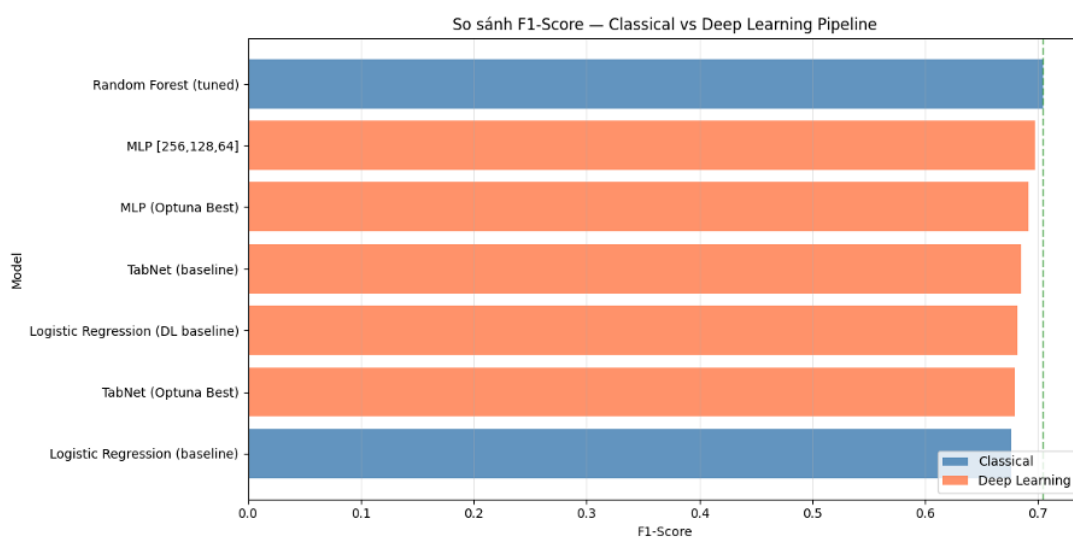
- **Pipeline truyền thống:**
 - Logistic Regression (baseline)
 - Random Forest (tuned)
- **Pipeline học sâu:**
 - Logistic Regression (baseline học sâu)
 - MLP (cấu hình ban đầu)
 - MLP (cấu hình tốt nhất của Optuna)
 - TabNet (cấu hình ban đầu)
 - TabNet (cấu hình tốt nhất của Optuna)

Nhóm tiến hành tổng hợp kết quả từ các lần đánh giá trên bộ dữ liệu test, thu được bảng kết quả như sau:

Bảng 6.1: Kết quả so sánh hiệu năng các mô hình

Mô hình	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression (baseline)	0.807350	0.565706	0.839607	0.675964	0.905146
Random Forest (tuned)	0.833862	0.613528	0.826347	0.704210	0.917706
Logistic Regression (DL baseline)	0.804500	0.571600	0.844300	0.681700	0.903800
MLP (baseline)	0.832600	0.632200	0.776500	0.697000	0.906900
MLP (Optuna)	0.810100	0.578600	0.860400	0.691900	0.910200
TabNet (baseline)	0.811800	0.585600	0.824300	0.684700	0.903400
TabNet (Optuna)	0.813600	0.592200	0.796200	0.679200	0.901300

Xét riêng về kết quả hai hướng tiếp cận, nhóm nhận thấy rằng:



Hình 6.1: Biểu đồ so sánh F1-Score giữa các mô hình

Ở pipeline truyền thống, quy trình xử lý dữ liệu, tạo đặc trưng, huấn luyện baseline, tuning và lựa chọn mô hình cuối cùng được tổ chức mạch lạc. Kết quả từ lần so sánh này cho thấy các mô hình cổ điển, đặc biệt là Logistic Regression và Random Forest, hoạt động rất cạnh tranh trên dữ liệu bảng sau khi được tiền xử lý tốt. Điều này phù hợp với đặc điểm của bộ dữ liệu Adult: số lượng mẫu vừa phải, nhiều biến categorical và cấu trúc tương đối ổn định.

Ở pipeline học sâu, MLP và TabNet đem lại góc nhìn mở rộng về khả năng học quan hệ phi tuyến. Tuy nhiên, lợi thế của học sâu trên bộ dữ liệu này không thực sự áp đảo. MLP sau Optuna có cải thiện so với cấu hình cơ bản, nhưng chênh lệch với baseline không lớn. TabNet là hướng thú vị về mặt kiến trúc, song ở thực nghiệm hiện tại cần nhiều công sức tuning hơn để đạt hiệu quả mong muốn.

Cuối cùng, nhóm tổng kết và đưa ra mô hình tốt nhất giữa hai hướng tiếp cận là **mô hình Random Forest** của pipeline học máy truyền thống.

```
Saved: /YourWorkspace/ml-project-group10/results/final\_summary\_all\_models.csv

=== PIPELINE HOÀN THÀNH ===
Tất cả kết quả đã được lưu tại: /YourWorkspace/ml-project-group10/results
Features đã lưu tại           : /YourWorkspace/ml-project-group10/features
Model cuối cùng lưu tại      : /YourWorkspace/ml-project-group10/models

=== MÔ HÌNH TỐT NHẤT ===
Pipeline : Classical
Model    : Random Forest (tuned)
F1-Score : 0.7042099507927829
ROC-AUC  : 0.9177055082982428
Accuracy : 0.8338622172177296
```

Hình 6.2: Mô hình học máy tốt nhất

6.2 Ưu và nhược điểm của từng hướng tiếp cận

Pipeline học máy truyền thống

- **Ưu điểm:** dễ diễn giải, huấn luyện nhanh, dễ tái lập và đặc biệt hiệu quả trên dữ liệu bảng đã qua xử lý tốt.
- **Nhược điểm:** khả năng biểu diễn phi tuyến bị giới hạn hơn nếu không chọn mô hình phù hợp hoặc không engineering thêm đặc trưng.

Pipeline học sâu

- **Ưu điểm:** linh hoạt hơn về mặt biểu diễn, có thể khai thác các quan hệ phi tuyến sâu hơn; dễ mở rộng nếu bài toán hoặc quy mô dữ liệu phức tạp hơn trong tương lai.
- **Nhược điểm:** chi phí huấn luyện và tuning cao hơn, kết quả nhạy hơn với siêu tham số, khó diễn giải hơn và trên bộ dữ liệu hiện tại chưa chứng minh được lợi thế vượt trội rõ ràng.

6.3 Kết luận chung

Dựa trên toàn bộ quá trình cài đặt và thực nghiệm, nhóm đi đến kết luận rằng bộ dữ liệu Adult Census Income là một ví dụ điển hình cho trường hợp **mô hình truyền thống mạnh vẫn là lựa chọn rất tốt**. Học sâu vẫn có giá trị nghiên cứu và cho thấy khả năng cải thiện nhẹ sau tuning, nhưng lợi ích thu được cần được cân nhắc cùng với độ phức tạp bổ sung trong cài đặt và huấn luyện.

Nếu mục tiêu là xây dựng một hệ thống dự đoán gọn nhẹ, dễ giải thích và có hiệu năng ổn định, pipeline truyền thống là lựa chọn phù hợp. Nếu mục tiêu là mở rộng nghiên cứu, thử nghiệm các kiến trúc hiện đại cho dữ liệu bảng và chấp nhận chi phí tuning cao hơn, pipeline học sâu là hướng phát triển tiếp theo đáng cân nhắc.